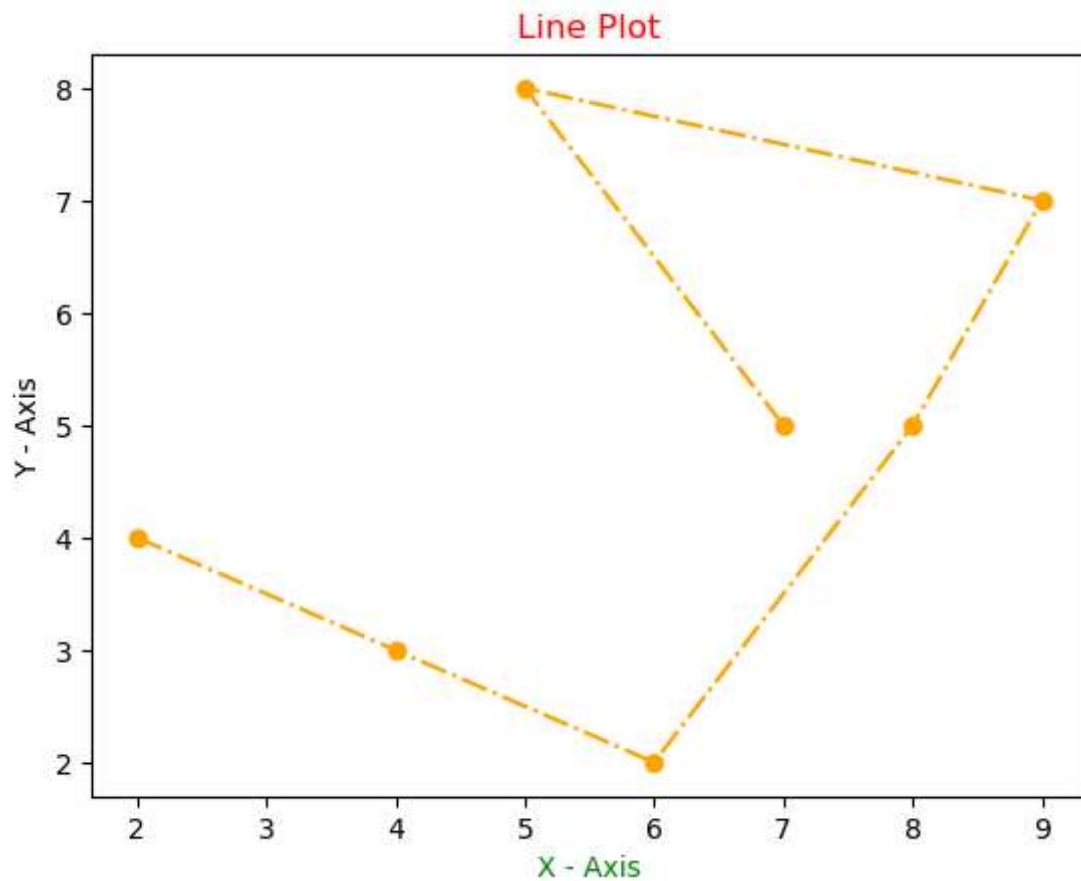
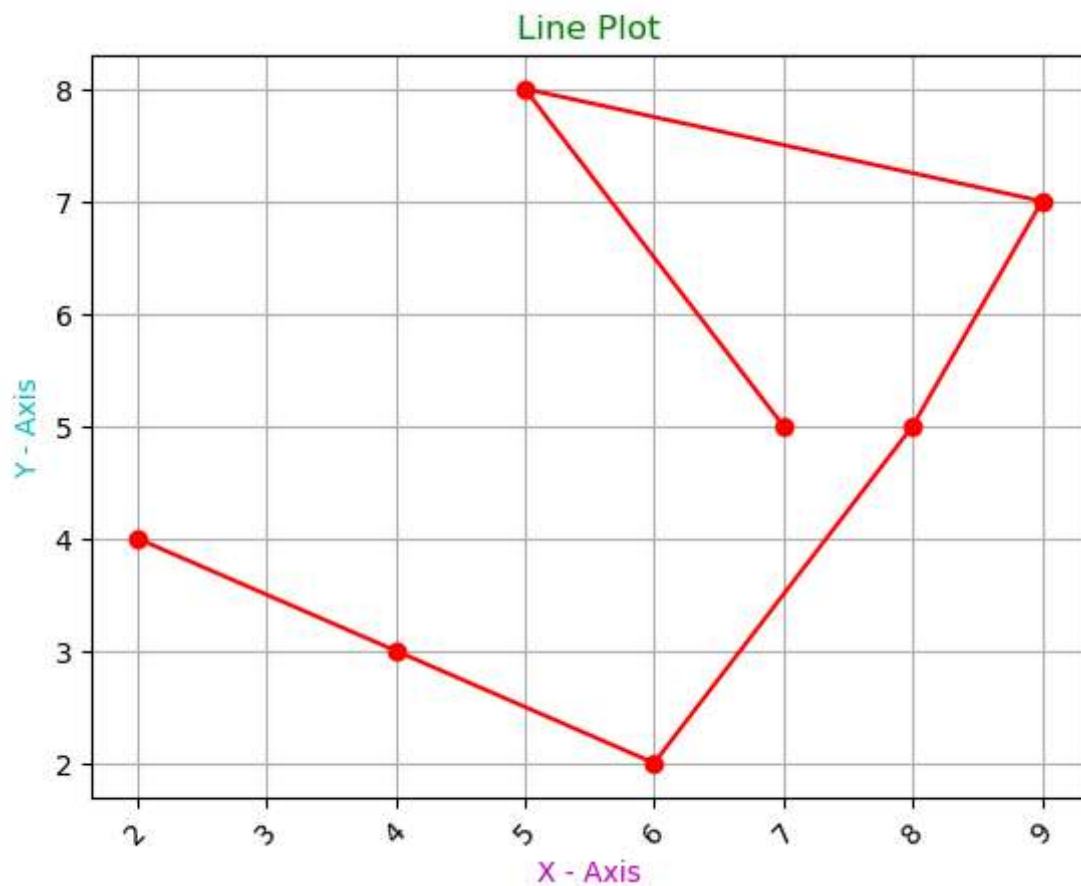


```
In [1]: import matplotlib.pyplot as plt
```

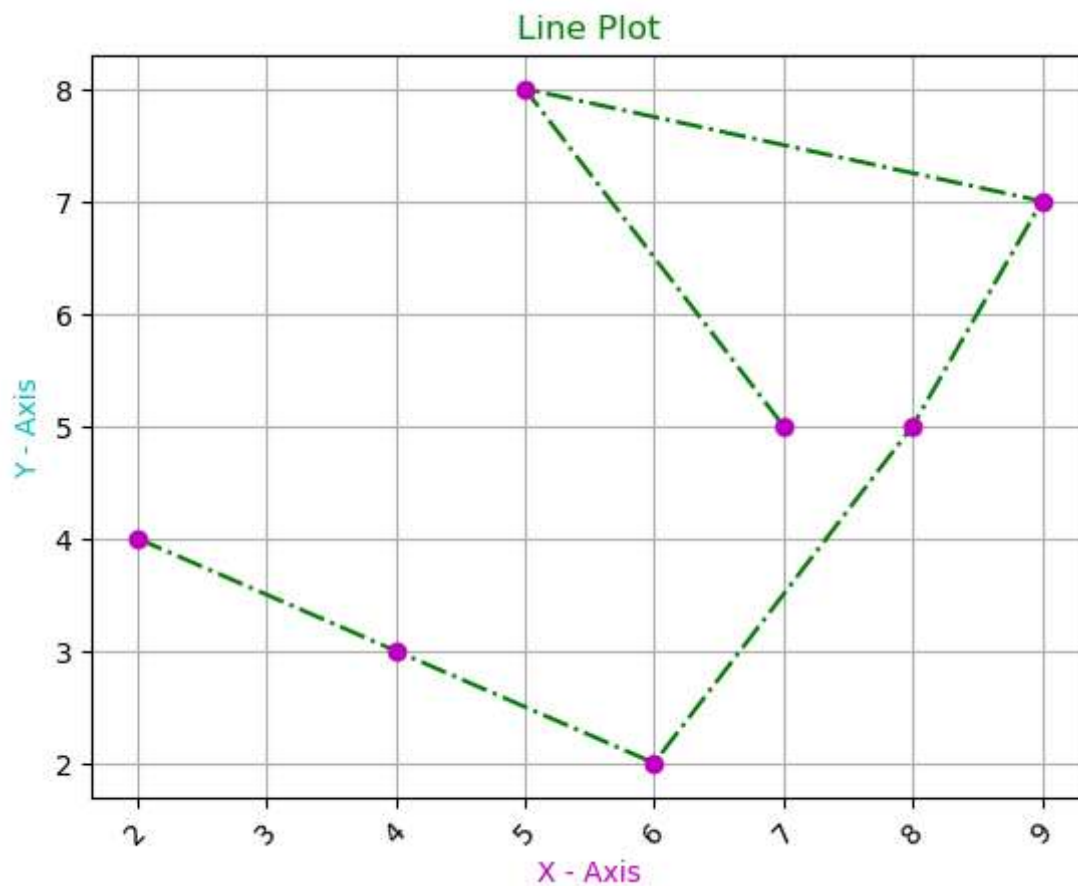
```
In [2]: x = [2,4,6,8,9,5,7]
y = [4,3,2,5,7,8,5]
plt.plot(x,y,'o-.',color='orange')
plt.title('Line Plot',color='r')
plt.xlabel('X - Axis',color='g')
plt.ylabel('Y - Axis')
plt.show()
```



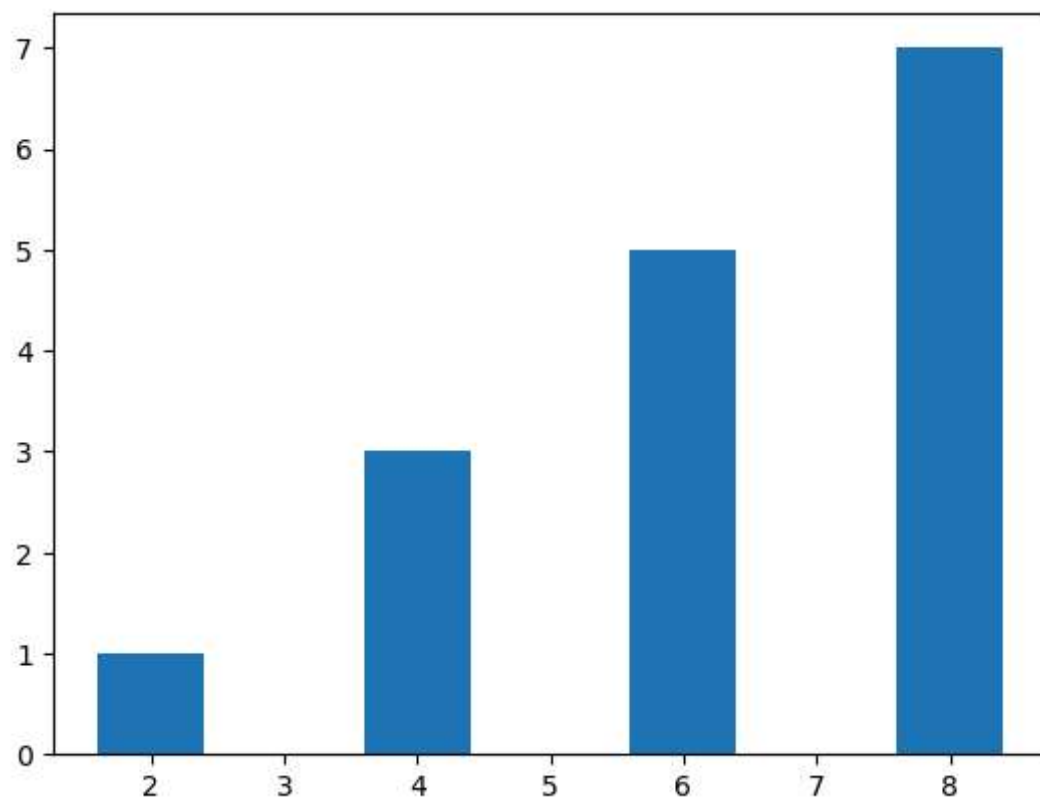
```
In [3]: x = [2,4,6,8,9,5,7]
y = [4,3,2,5,7,8,5]
plt.plot(x,y,'or-')
plt.title('Line Plot',color='g')
plt.xlabel('X - Axis',color='m')
plt.ylabel('Y - Axis',color='c')
plt.xticks(rotation=45)
plt.grid()
plt.show()
```



```
In [4]: x = [2,4,6,8,9,5,7]
y = [4,3,2,5,7,8,5]
plt.plot(x,y,'g-.')
plt.plot(x,y,'om')
plt.title('Line Plot',color='g')
plt.xlabel('X - Axis',color='m')
plt.ylabel('Y - Axis',color='c')
plt.xticks(rotation=45)
plt.grid()
plt.show()
```

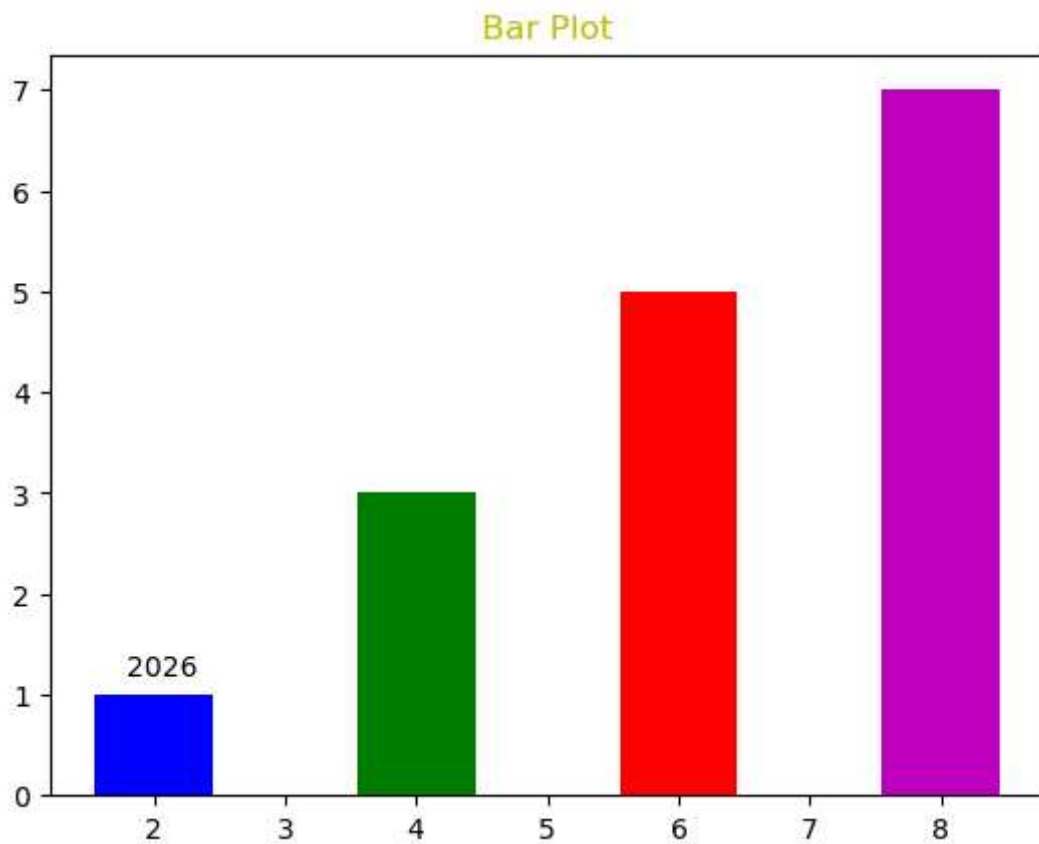


```
In [5]: x = [2,4,6,8]
y = [1,3,5,7]
plt.bar(x,y)
plt.show()
```

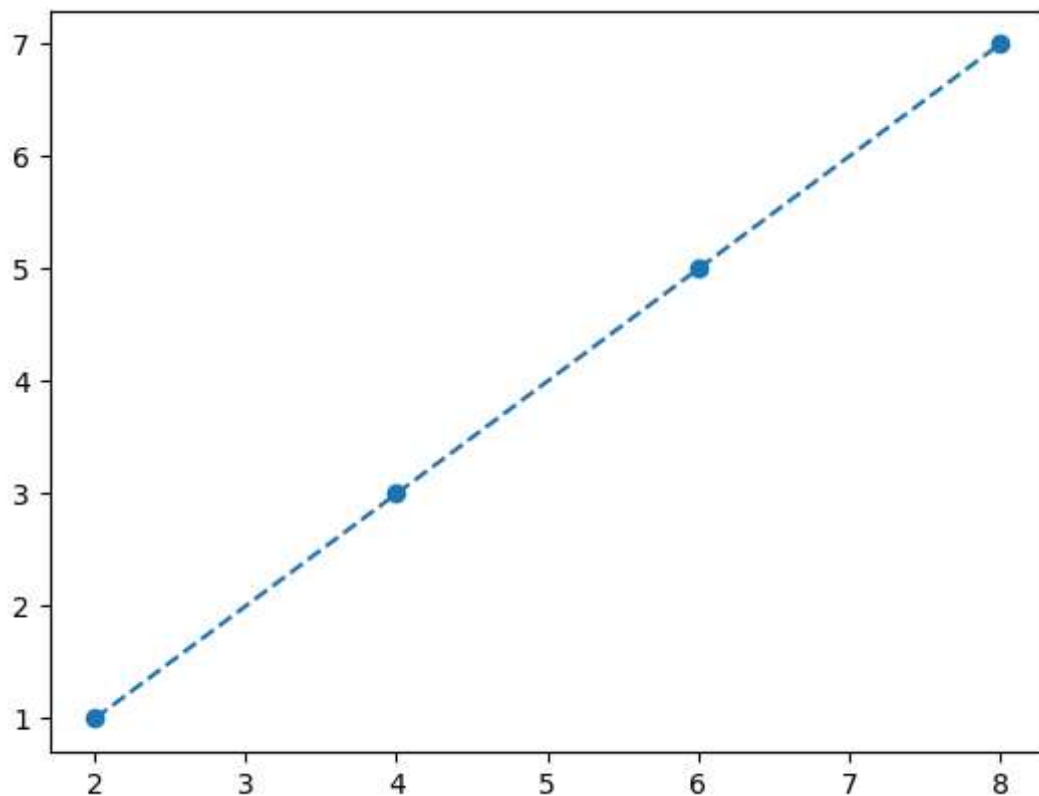


```
In [6]: x = [2,4,6,8]
y = [1,3,5,7]
```

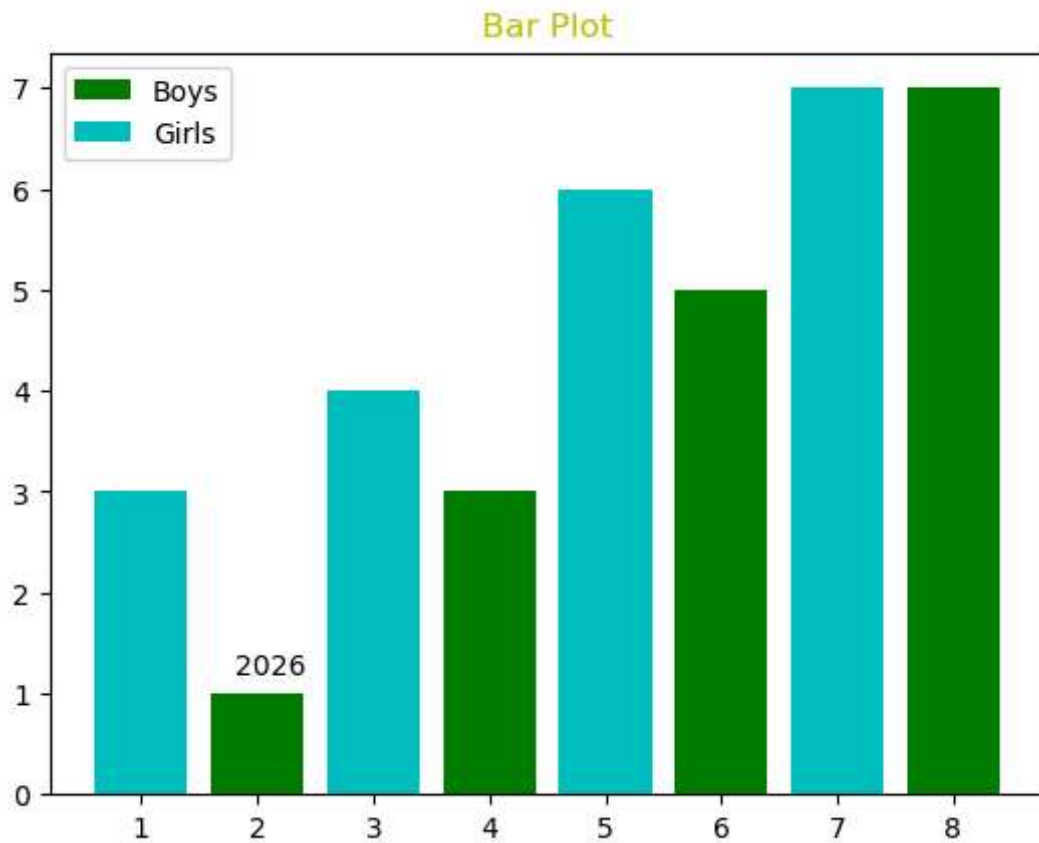
```
plt.bar(x,y,color=['b','g','r','m'],width=0.9)
plt.title('Bar Plot',color='y')
plt.annotate("2026",xy=(1.8,1.2))
plt.show()
plt.plot(x,y,'o--')
```



Out[6]: [

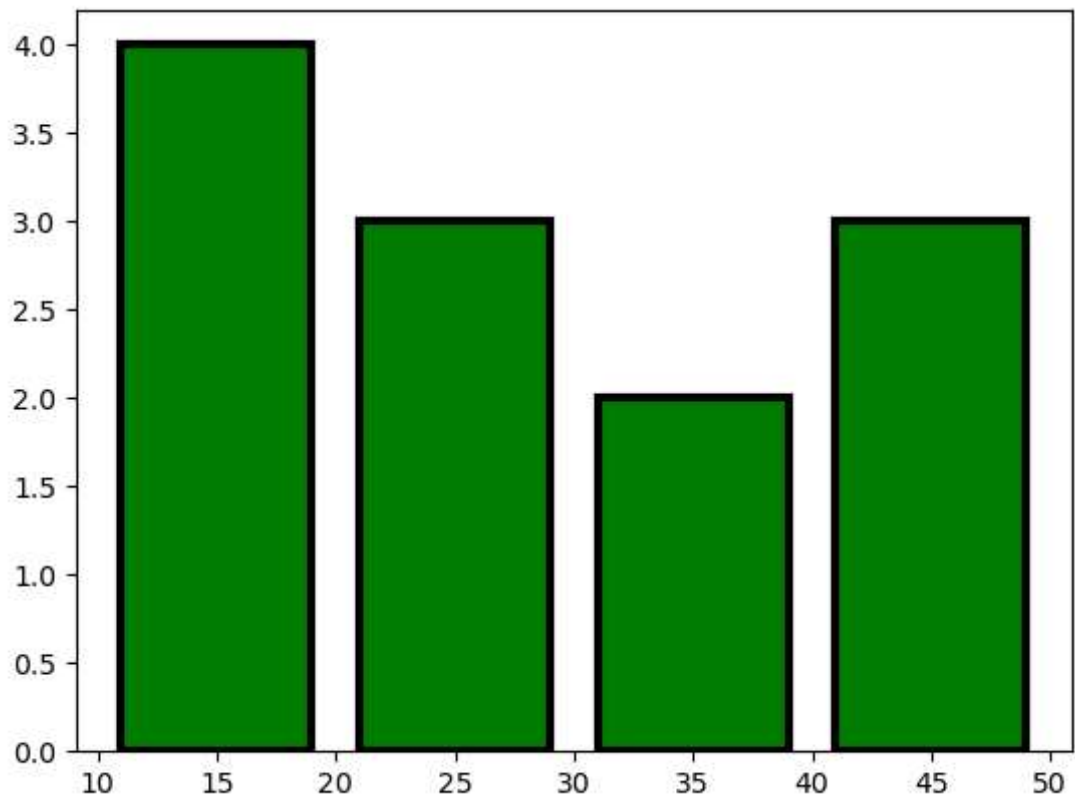


```
In [7]: x = [2,4,6,8]
y = [1,3,5,7]
plt.bar(x,y,color='g',width=0.8,label='Boys')
a = [1,3,5,7]
b = [3,4,6,7]
plt.bar(a,b,label = 'Girls',color='c')
plt.title('Bar Plot',color='y')
plt.annotate("2026",xy=(1.8,1.2))
plt.legend()
plt.show()
```



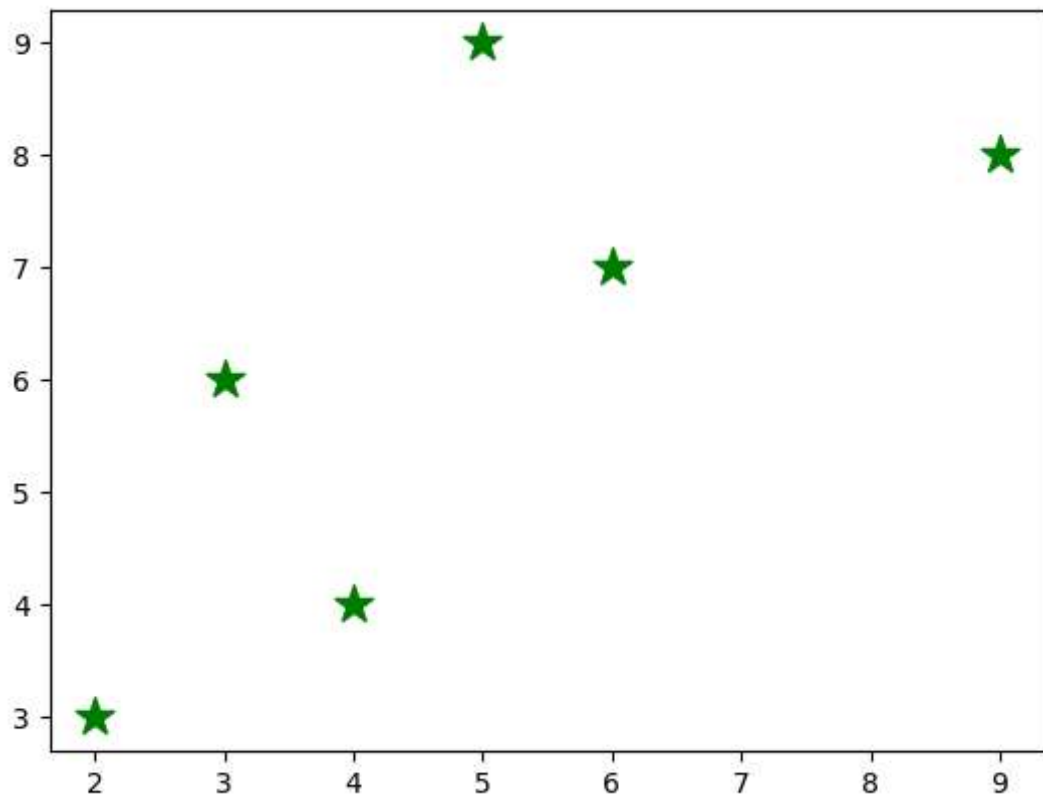
```
In [8]: #Histogram
ages = [12,15,17,18,20,23,21,30,33,49,48,50]
bins = [10,20,30,40,50]

plt.hist(ages,bins,rwidth=0.8,edgecolor='k',linewidth=3,color='g')
plt.show()
```



```
In [9]: #scatter plot
x = [2,4,3,6,9,5]
y = [3,4,6,7,8,9]

plt.scatter(x,y,s=200,color='g',marker='*')
plt.show()
```

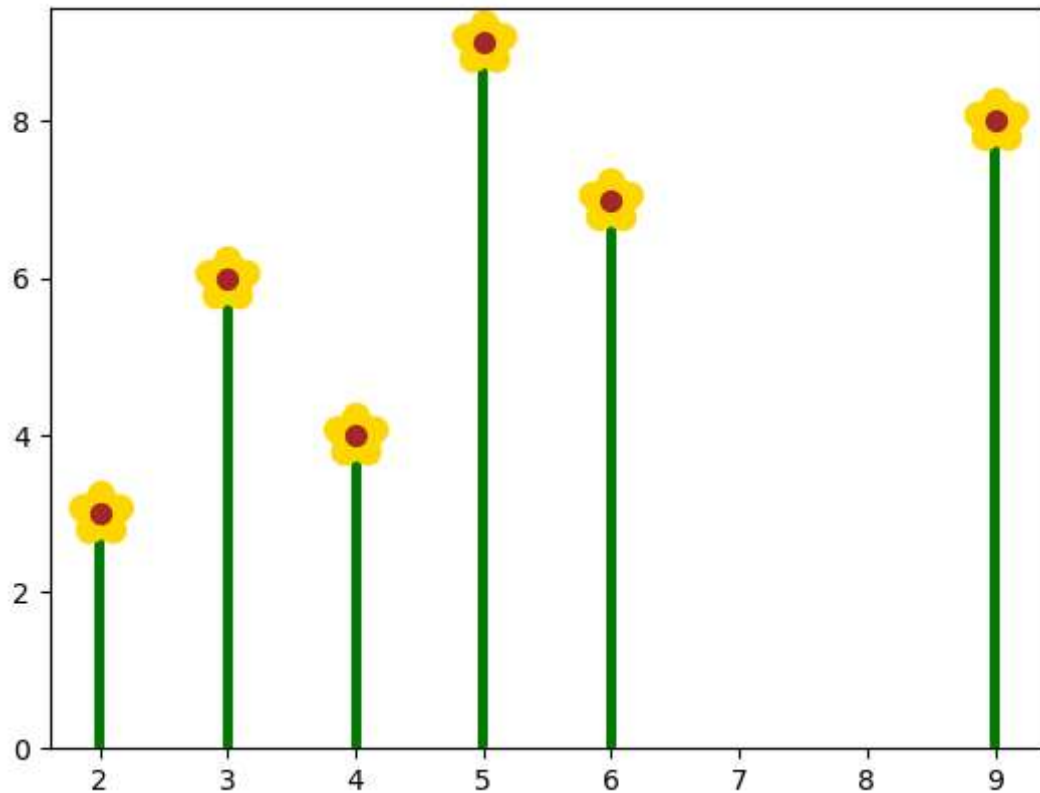


```
In [10]: #scatter plot
x = [2,4,3,6,9,5]
```

```

y = [3,4,6,7,8,9]
plt.bar(x,y,width=0.08,color='g')
plt.scatter(x,y,s=200,color='gold',marker='*',edgecolor='gold',linewidth=10)
plt.scatter(x,y,s=50,color='brown',marker='o')
plt.show()

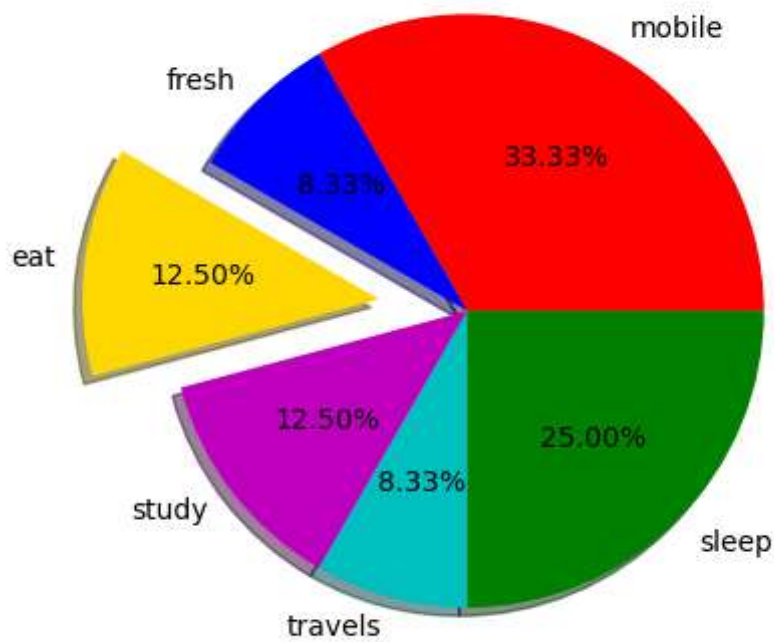
```



```

In [11]: #pie chart
kam = ['mobile', 'fresh', 'eat', 'study', 'travels', 'sleep']
slices = [8,2,3,3,2,6]
plt.pie(slices,labels=kam,colors=['r','b','gold','m','c','g'],
        startangle=0,autopct='%1.2f%%',shadow=True,explode=(0,0,0.3,0,0,0))
plt.show()

```



In []:

Seaborn

```
In [63]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [13]: iris = pd.read_csv(r"C:\Users\heyja\Downloads\iris\iris.data", header=None)
iris
```


Out[13]:

	0	1	2	3	4
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
In [14]: iris.columns = ['SL', 'SW', 'PL', 'PW', 'Flower Type']
iris
```

Out[14]:

	SL	SW	PL	PW	Flower Type
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
In [15]: for i in iris.columns:
          print(i,':','\n',iris[i].unique(),'\n')
```

SL :

```
[5.1 4.9 4.7 4.6 5.  5.4 4.4 4.8 4.3 5.8 5.7 5.2 5.5 4.5 5.3 7.  6.4 6.9
 6.5 6.3 6.6 5.9 6.  6.1 5.6 6.7 6.2 6.8 7.1 7.6 7.3 7.2 7.7 7.4 7.9]
```

SW :

```
[3.5 3.  3.2 3.1 3.6 3.9 3.4 2.9 3.7 4.  4.4 3.8 3.3 4.1 4.2 2.3 2.8 2.4
 2.7 2.  2.2 2.5 2.6]
```

PL :

```
[1.4 1.3 1.5 1.7 1.6 1.1 1.2 1.  1.9 4.7 4.5 4.9 4.  4.6 3.3 3.9 3.5 4.2
 3.6 4.4 4.1 4.8 4.3 5.  3.8 3.7 5.1 3.  6.  5.9 5.6 5.8 6.6 6.3 6.1 5.3
 5.5 6.7 6.9 5.7 6.4 5.4 5.2]
```

PW :

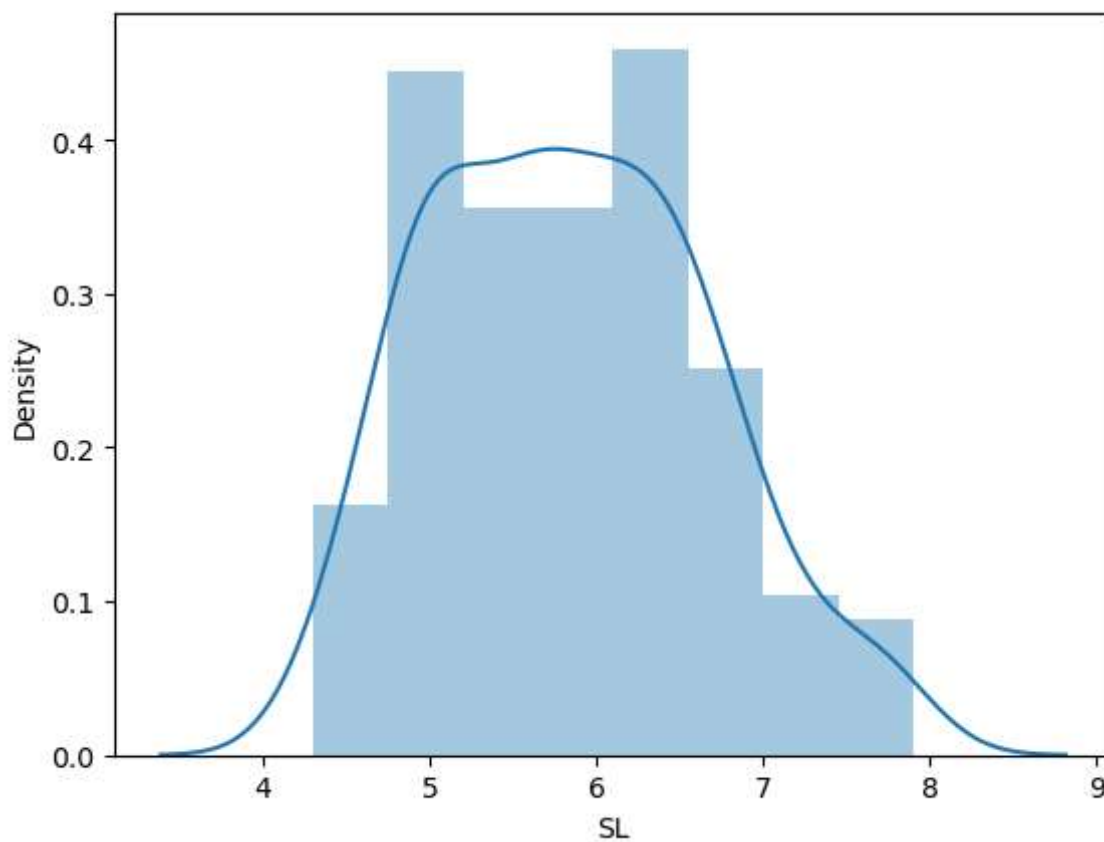
```
[0.2 0.4 0.3 0.1 0.5 0.6 1.4 1.5 1.3 1.6 1.  1.1 1.8 1.2 1.7 2.5 1.9 2.1
 2.2 2.  2.4 2.3]
```

Flower Type :

```
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

```
In [16]: import warnings
warnings.filterwarnings('ignore')
```

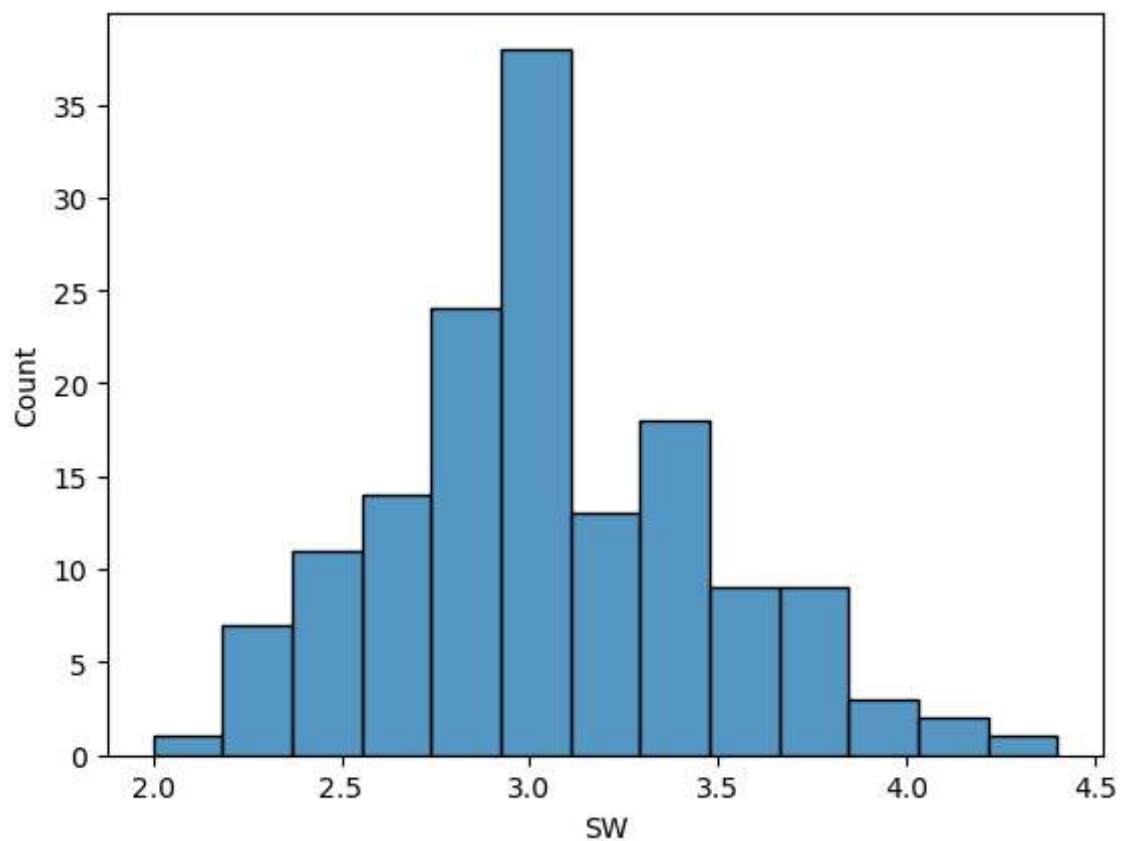
```
In [17]: #distribution plot
sns.distplot(iris.SL)
plt.show()
```



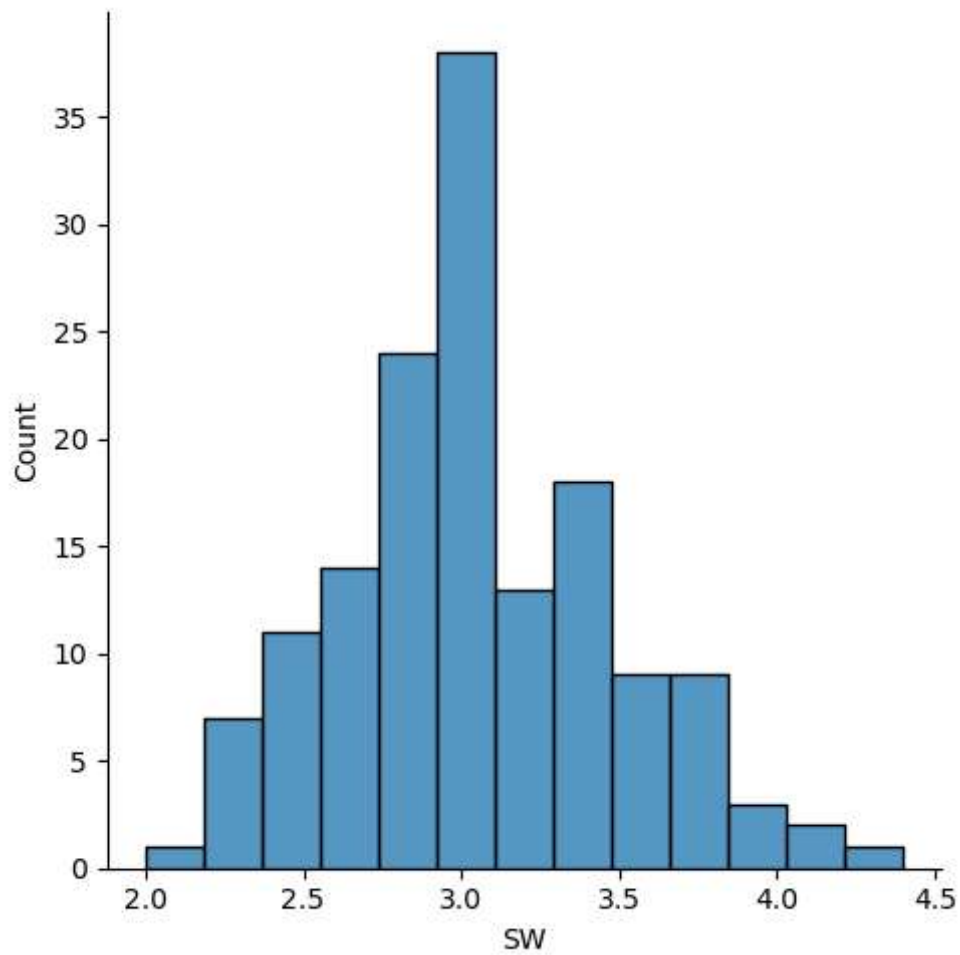
```
In [18]: iris.SW.value_counts()
```

```
Out[18]: SW
3.0      26
2.8      14
3.2      13
3.1      12
3.4      12
2.9      10
2.7       9
2.5       8
3.5       6
3.3       6
3.8       6
2.6       5
2.3       4
3.7       3
2.4       3
2.2       3
3.6       3
3.9       2
4.4       1
4.0       1
4.1       1
4.2       1
2.0       1
Name: count, dtype: int64
```

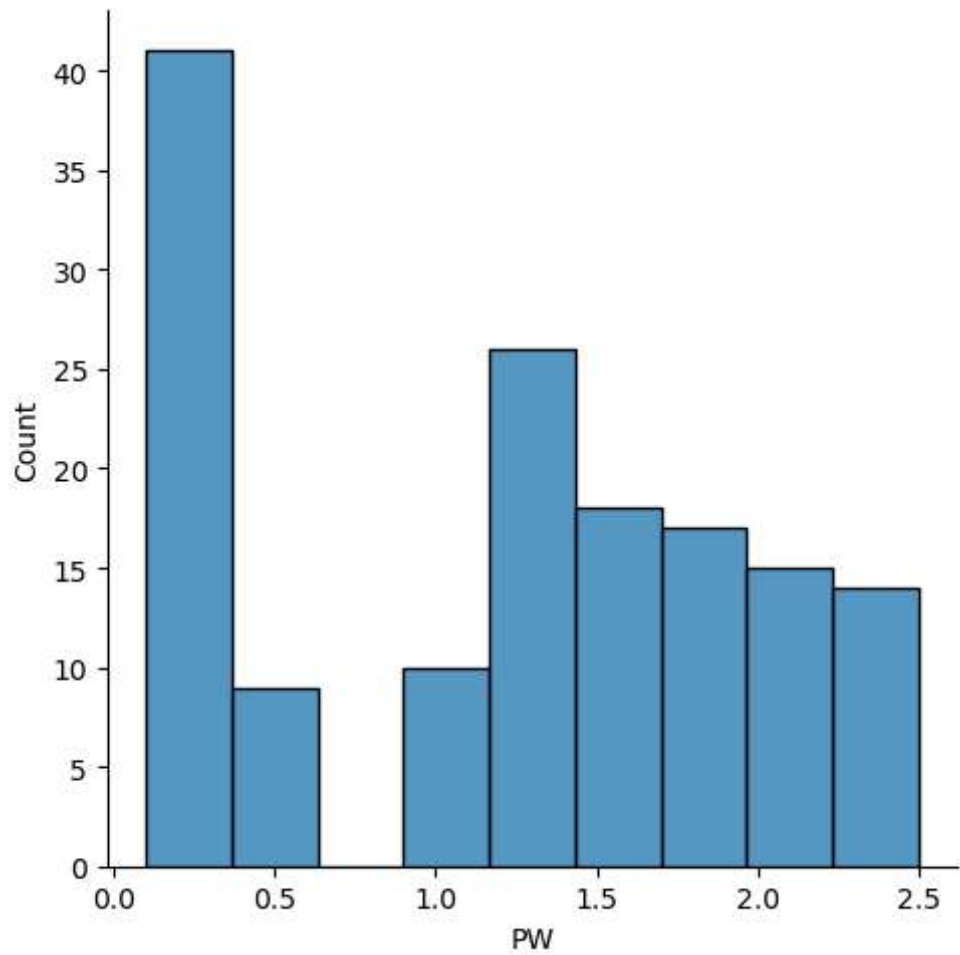
```
In [19]: sns.histplot(iris.SW)
plt.show()
```



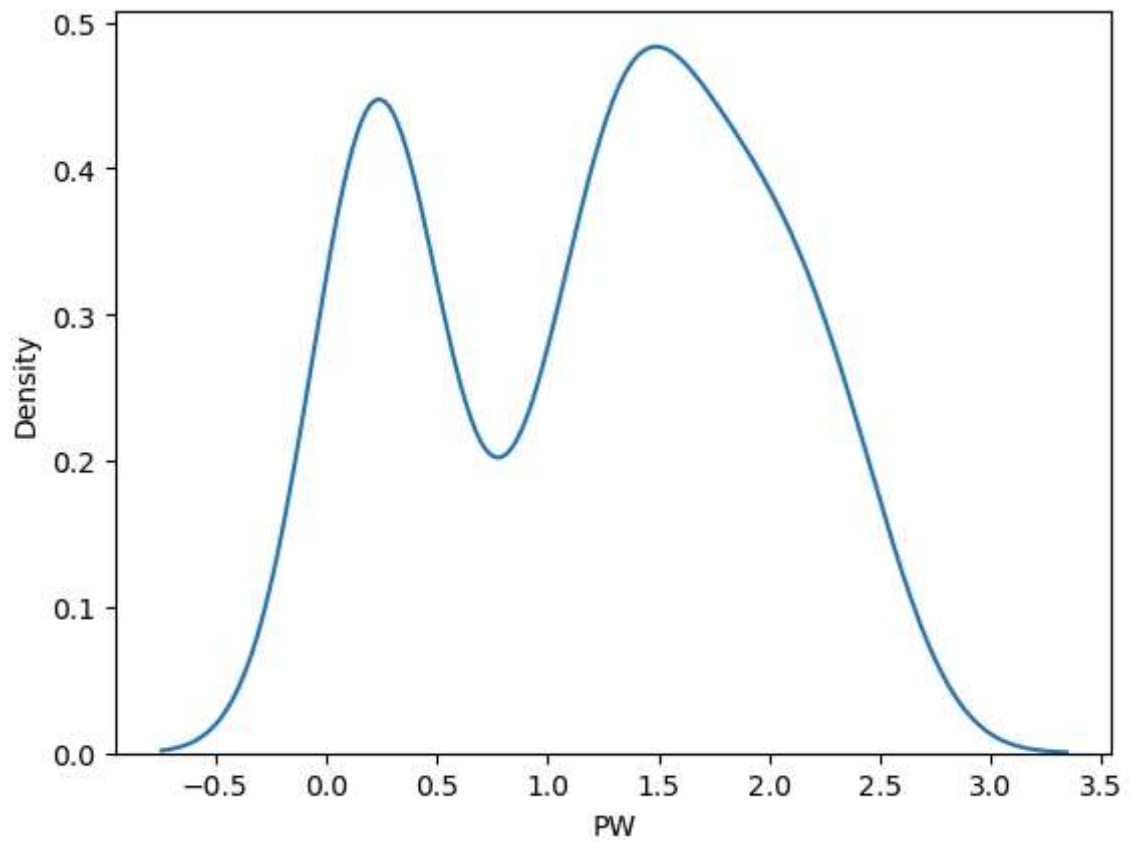
```
In [20]: sns.displot(iris.SW)
plt.show()
```



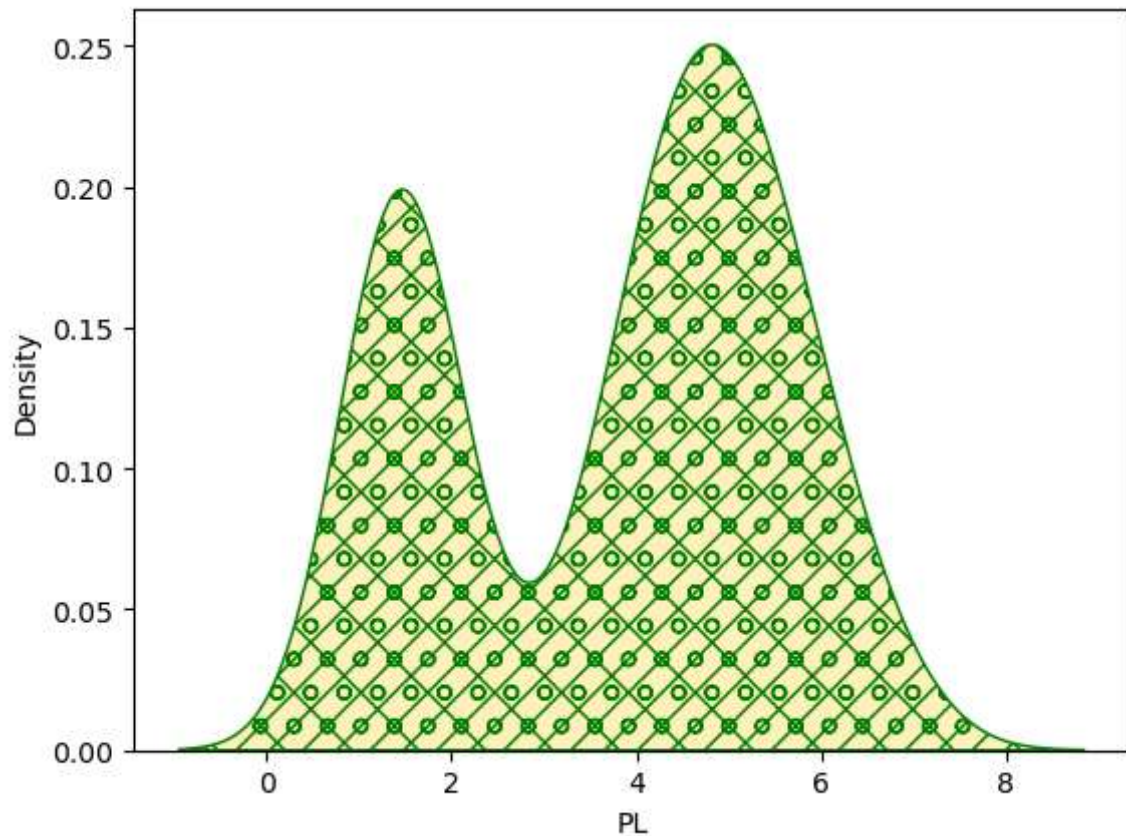
```
In [21]: sns.displot(iris.PW)
plt.show()
```



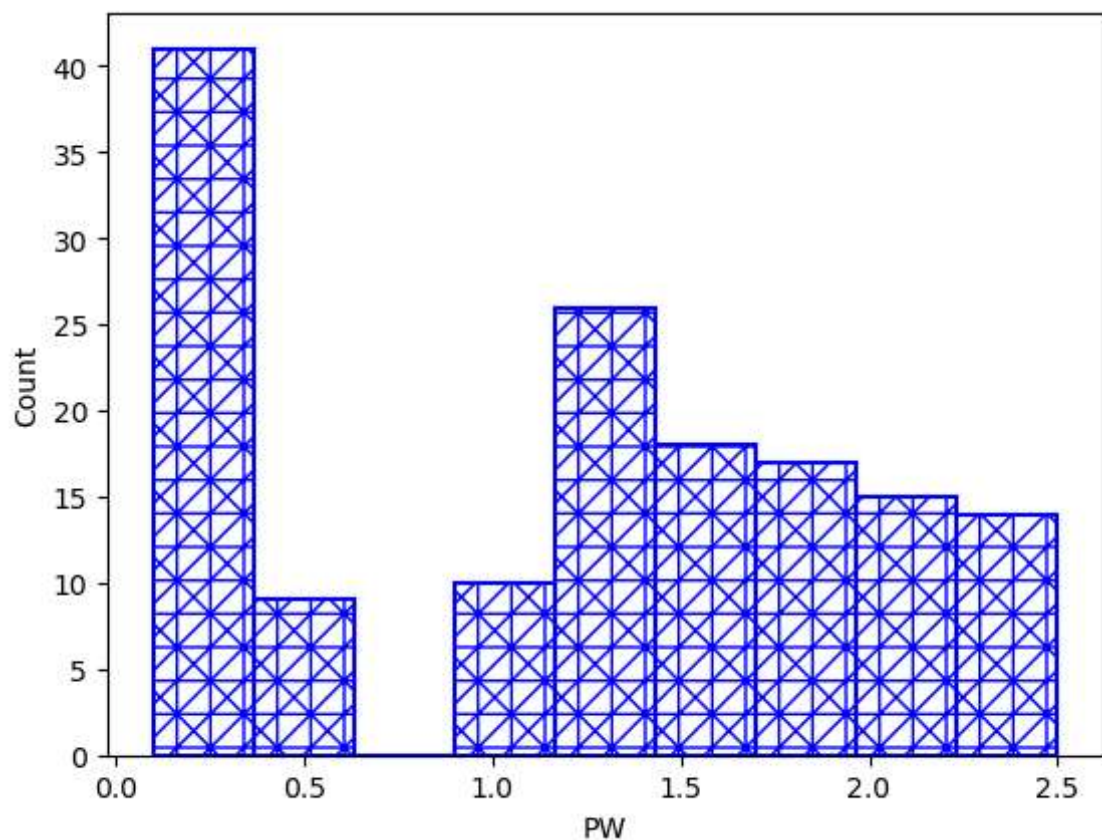
```
In [22]: sns.kdeplot(iris.PW)  
plt.show()
```



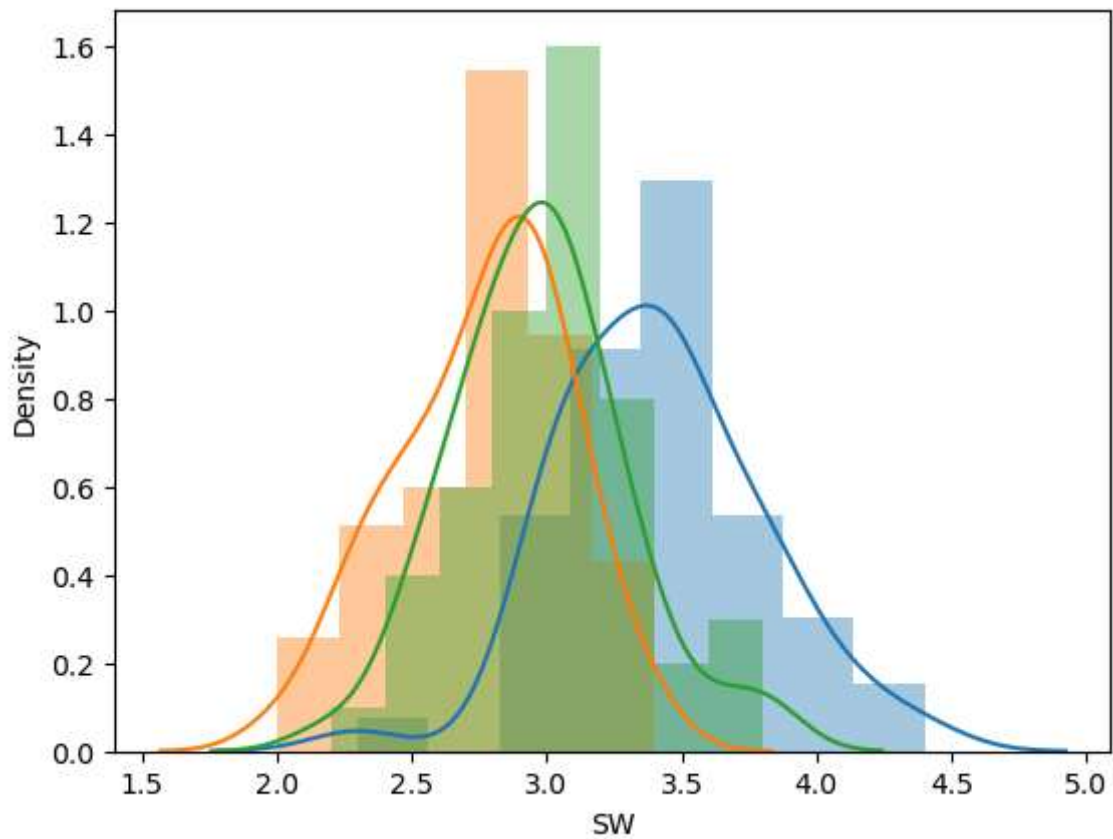
```
In [23]: sns.kdeplot(iris.PL,fill=True,color='gold',hatch='o//',edgecolor='g')
plt.show()
```



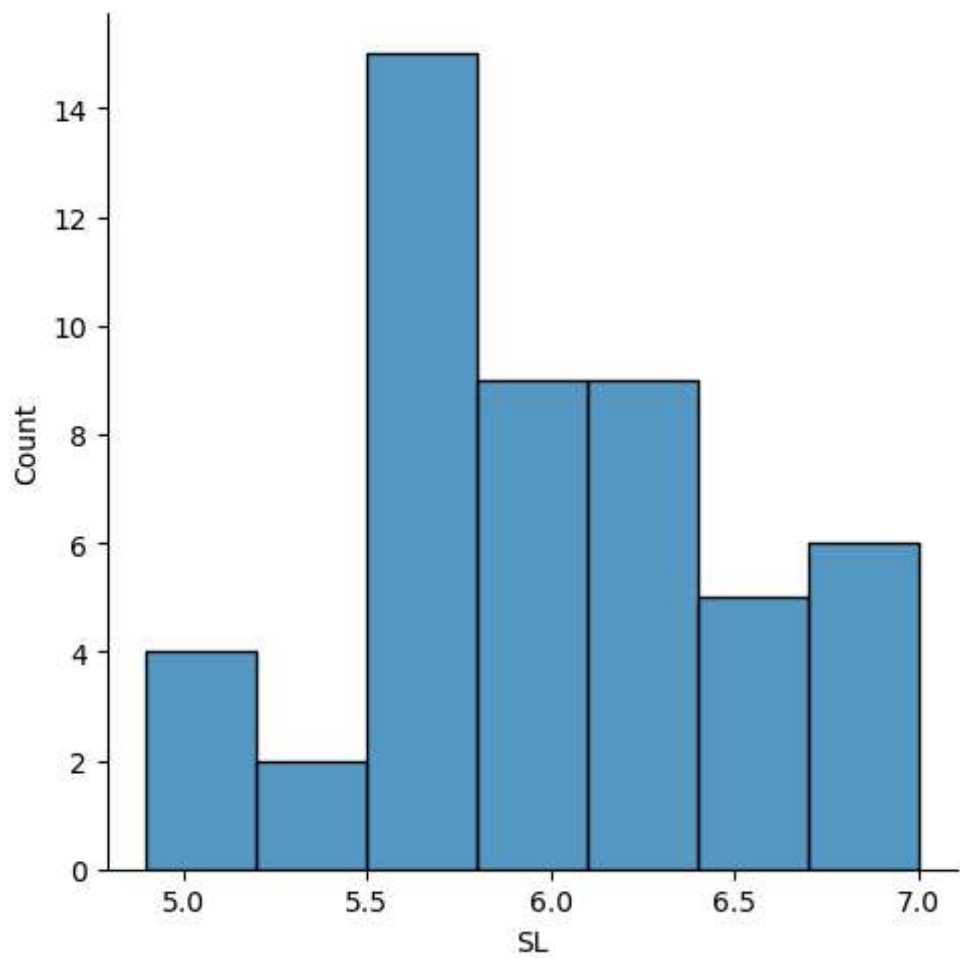
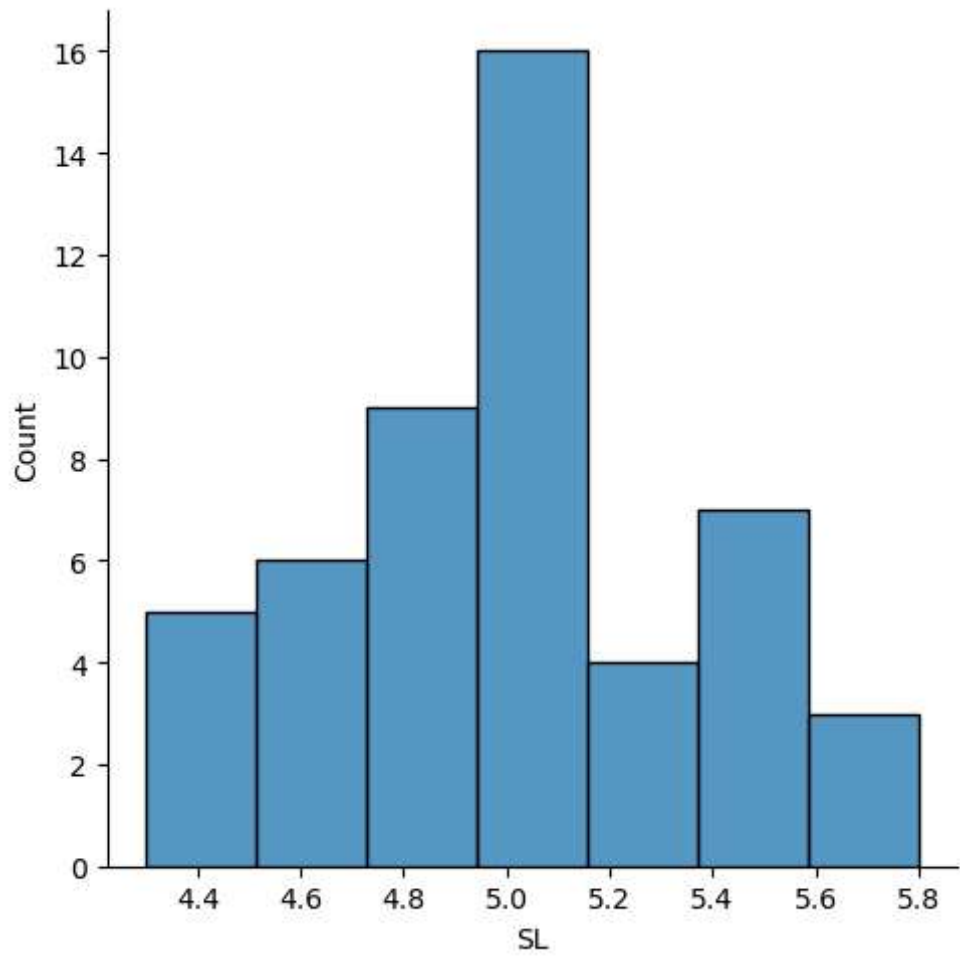
```
In [24]: sns.histplot(iris.PW,color='b',fill=False,hatch='\\+//')
plt.show()
```

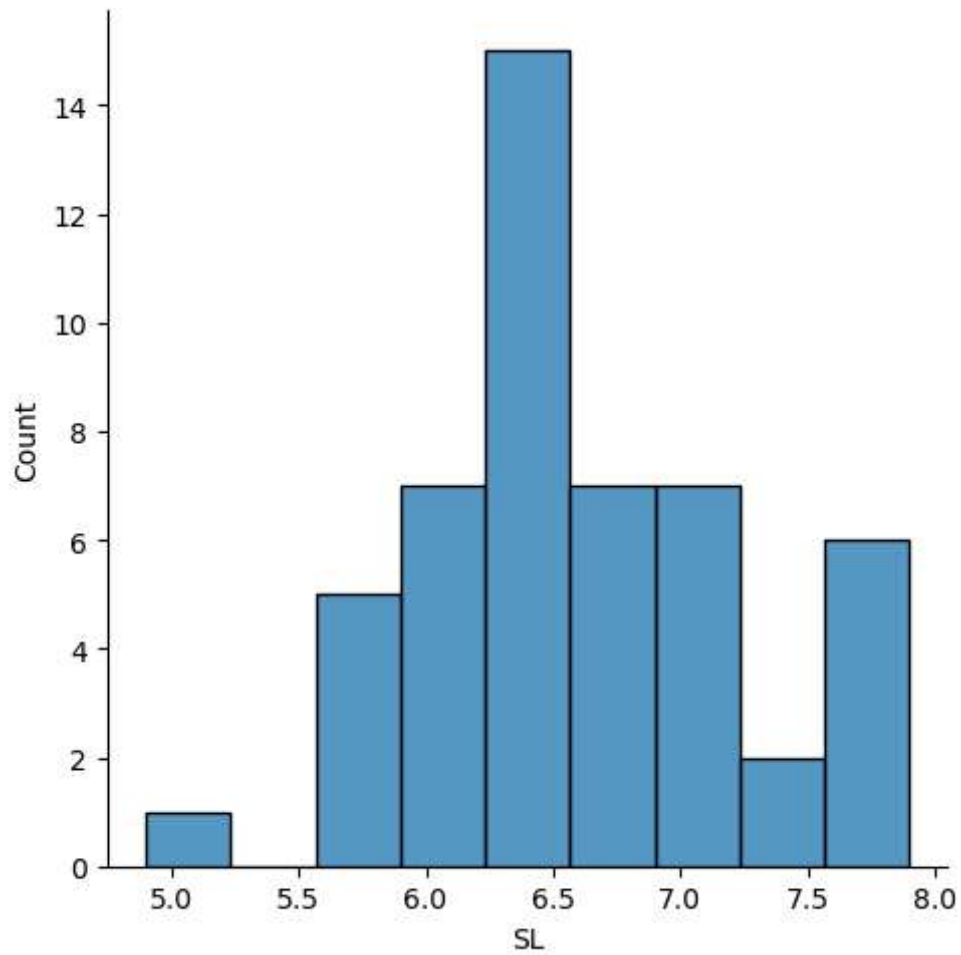


```
In [25]: sns.distplot(iris.SW[iris['Flower Type']=='Iris-setosa'])  
sns.distplot(iris.SW[iris['Flower Type']=='Iris-versicolor'])  
sns.distplot(iris.SW[iris['Flower Type']=='Iris-virginica'])  
plt.show()
```

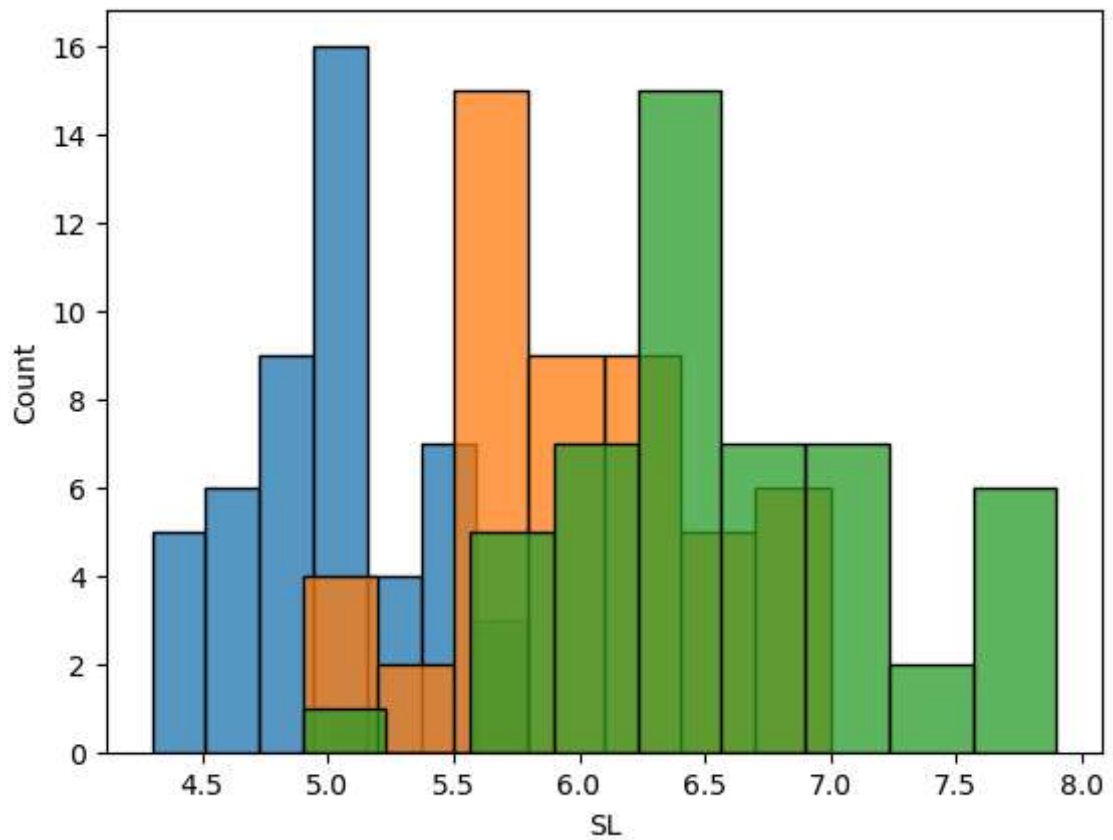


```
In [26]: sns.displot(iris.SL[iris['Flower Type']=='Iris-setosa'])  
sns.displot(iris.SL[iris['Flower Type']=='Iris-versicolor'])  
sns.displot(iris.SL[iris['Flower Type']=='Iris-virginica'])  
plt.show()
```

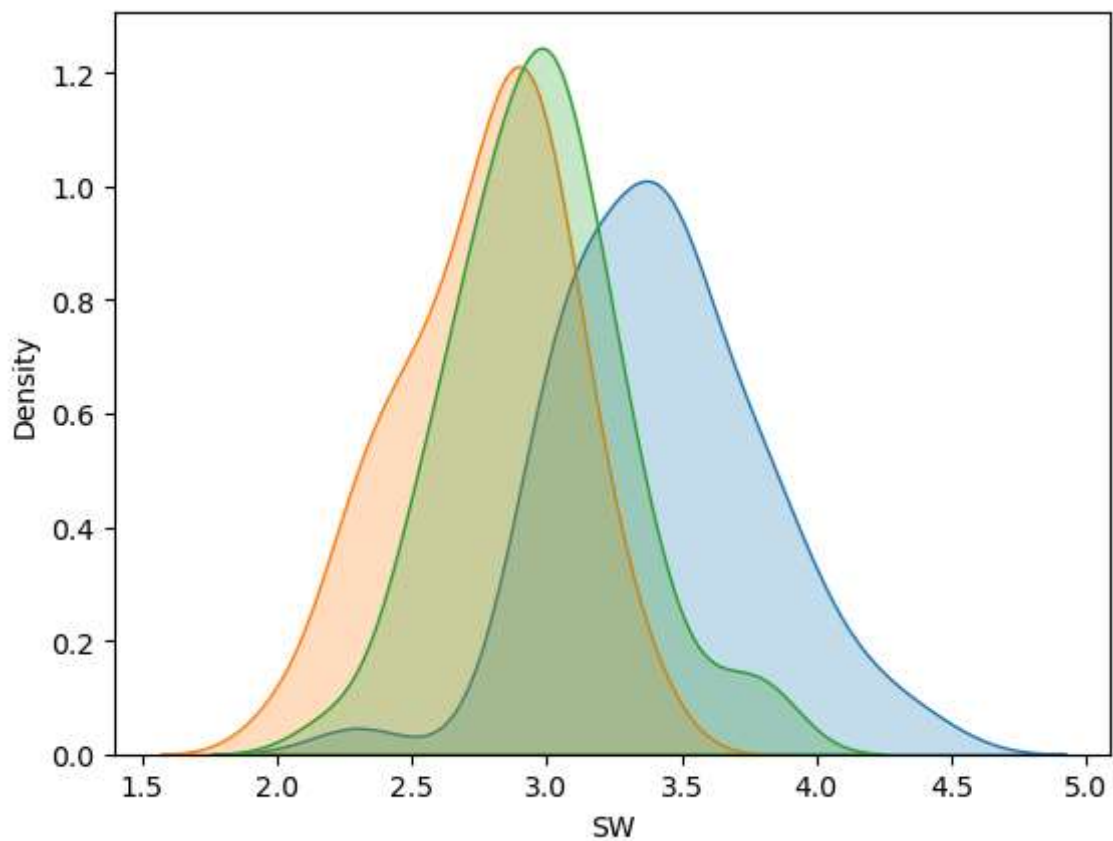




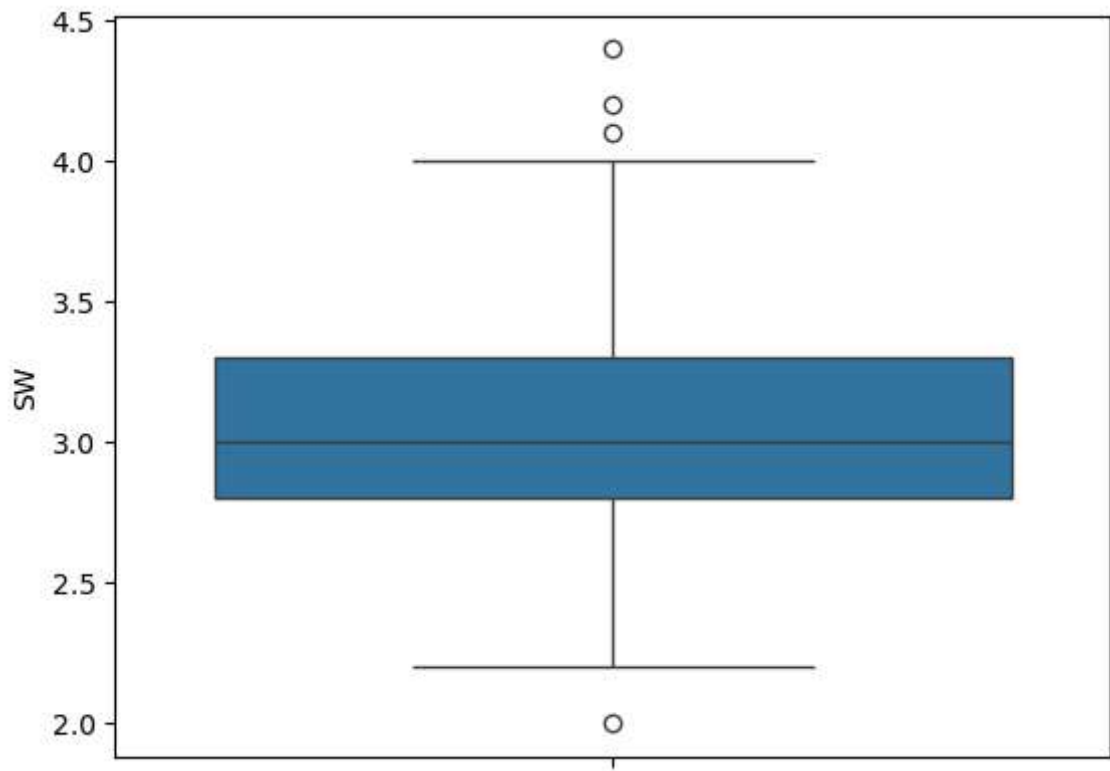
```
In [27]: sns.histplot(iris.SL[iris['Flower Type']=='Iris-setosa'])
sns.histplot(iris.SL[iris['Flower Type']=='Iris-versicolor'])
sns.histplot(iris.SL[iris['Flower Type']=='Iris-virginica'])
plt.show()
```



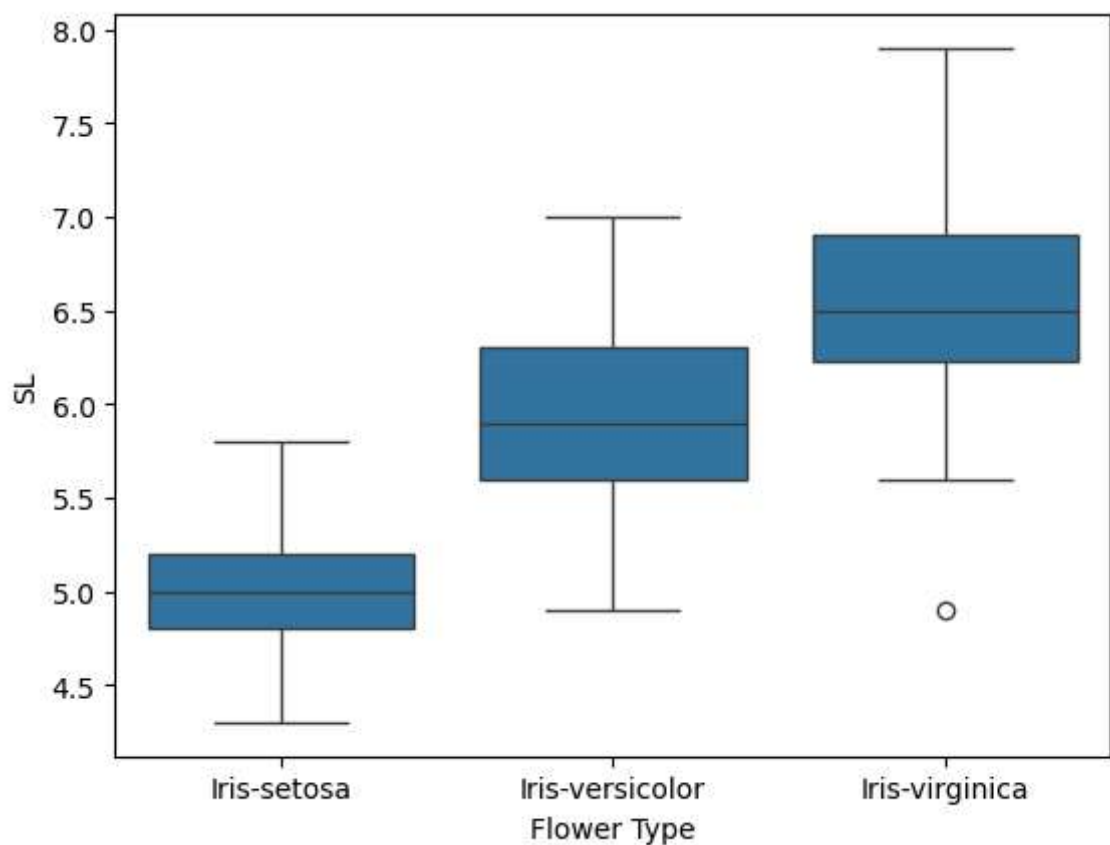
```
In [28]: sns.kdeplot(iris.SW[iris['Flower Type']=='Iris-setosa'],fill=True)
sns.kdeplot(iris.SW[iris['Flower Type']=='Iris-versicolor'],fill=True)
sns.kdeplot(iris.SW[iris['Flower Type']=='Iris-virginica'],fill=True)
plt.show()
```



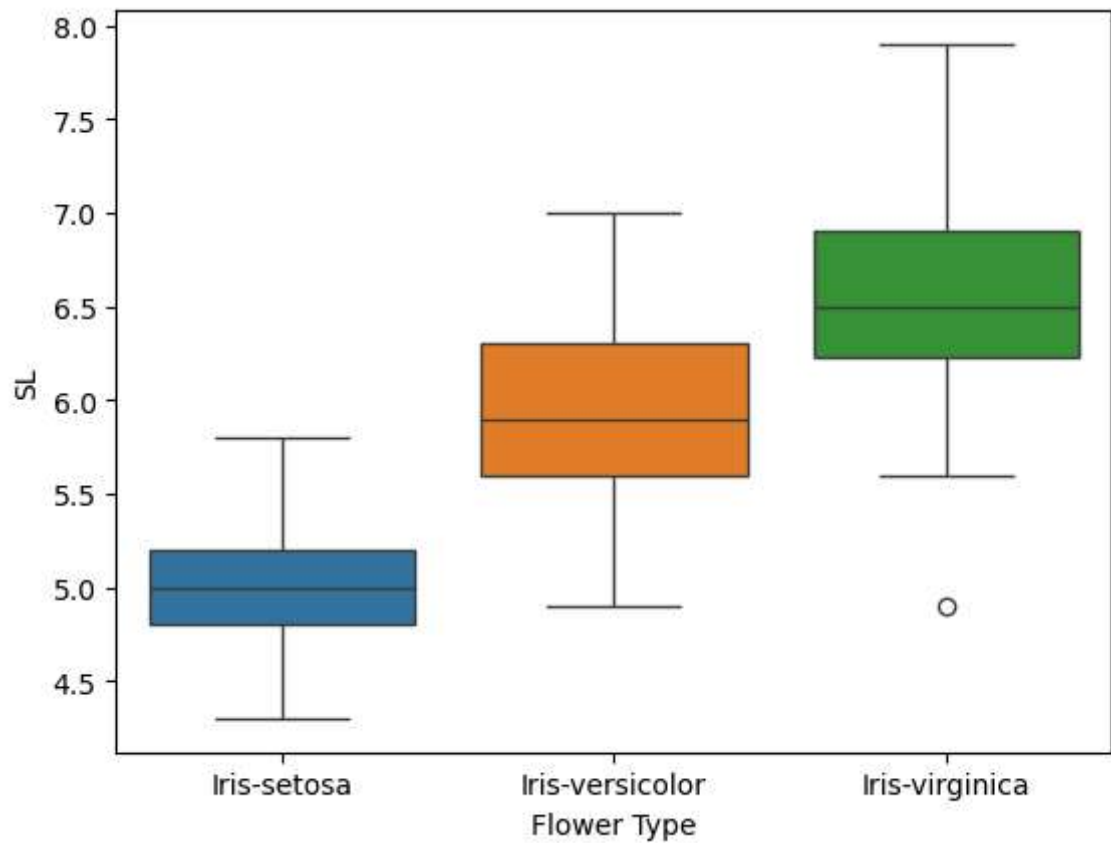
```
In [29]: sns.boxplot(iris.SW)
plt.show()
```



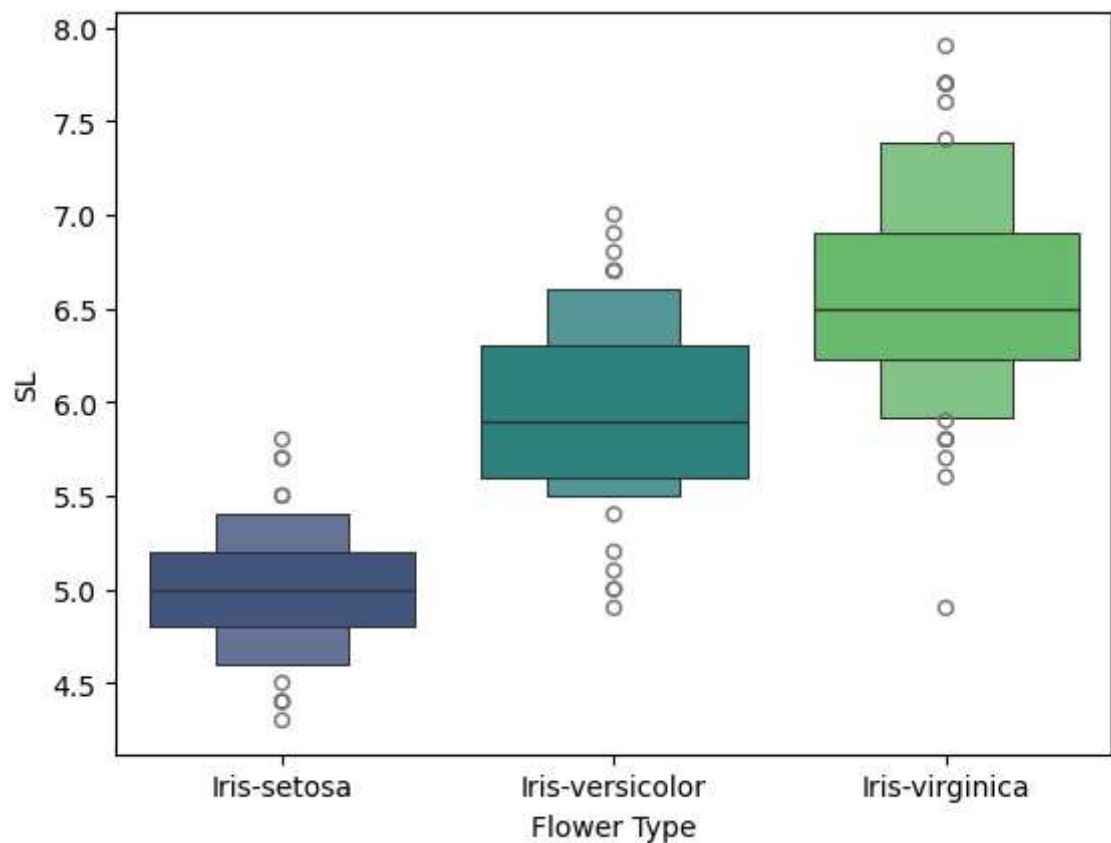
```
In [30]: sns.boxplot(x=iris['Flower Type'],y = iris.SL)  
plt.show()
```



```
In [31]: sns.boxplot(x=iris['Flower Type'],y = iris.SL,hue=iris['Flower Type'])  
plt.show()
```

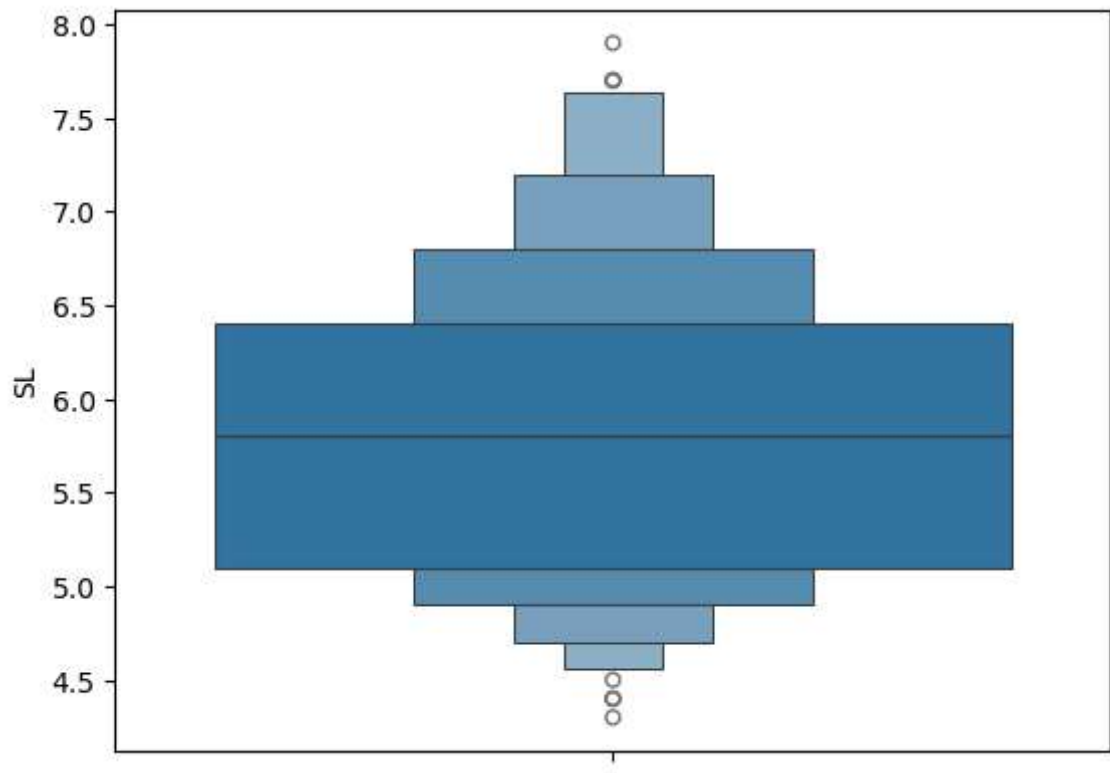


```
In [32]: #boxen plot
sns.boxenplot(x=iris['Flower Type'], y = iris.SL, palette='viridis')
plt.show()
```



```
In [33]: sns.boxenplot(iris.SL)
```

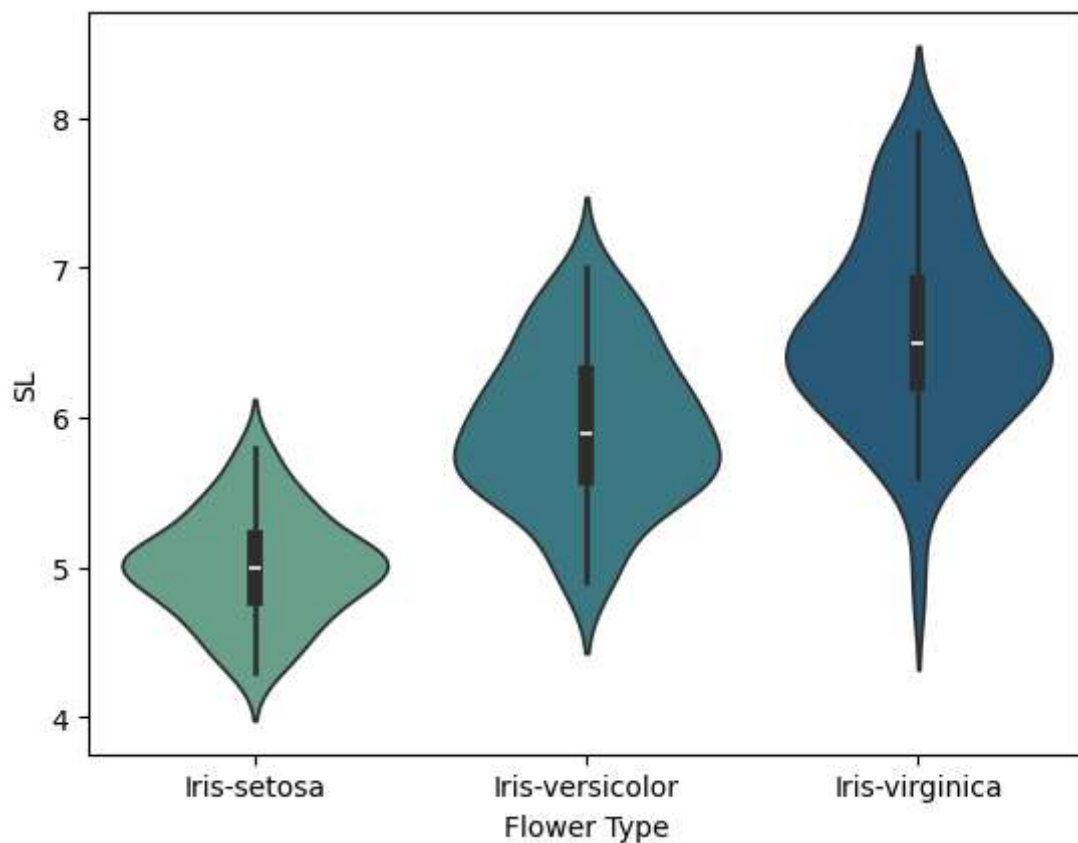
```
Out[33]: <Axes: ylabel='SL'>
```



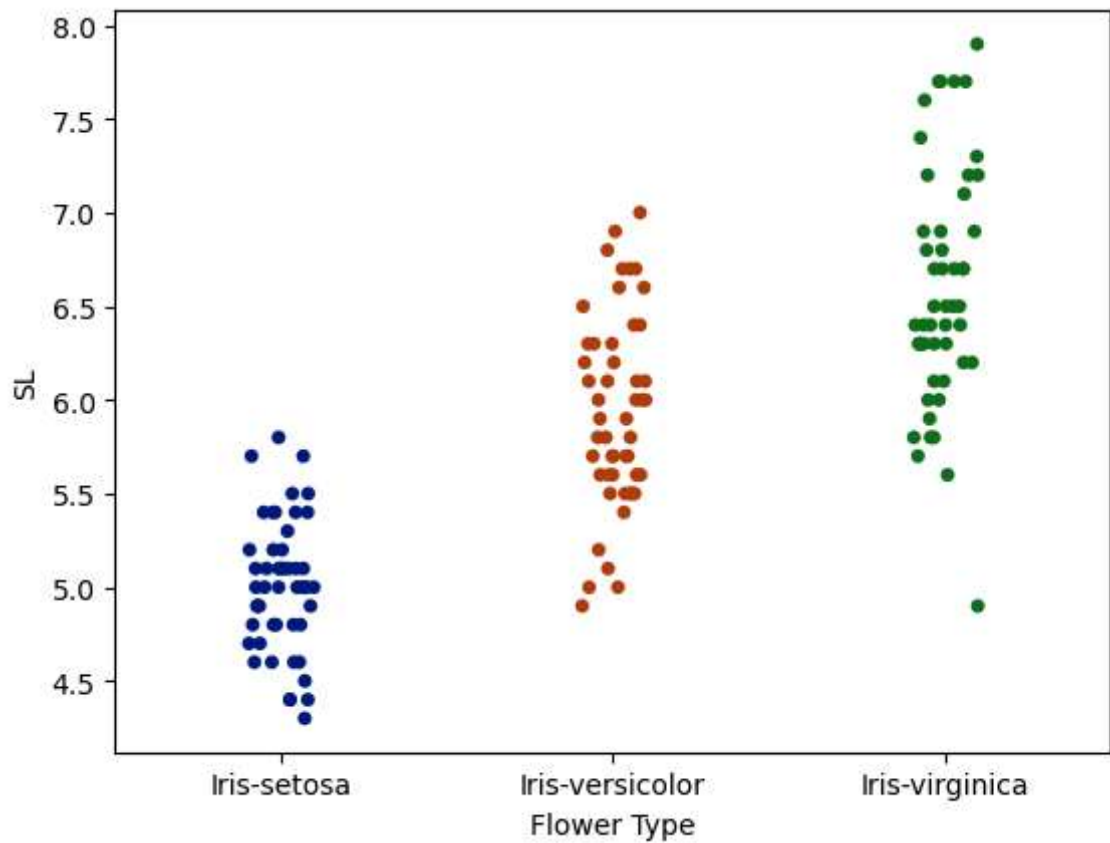
```
In [34]: np.percentile(iris.SL,6.25)
```

```
Out[34]: 4.7
```

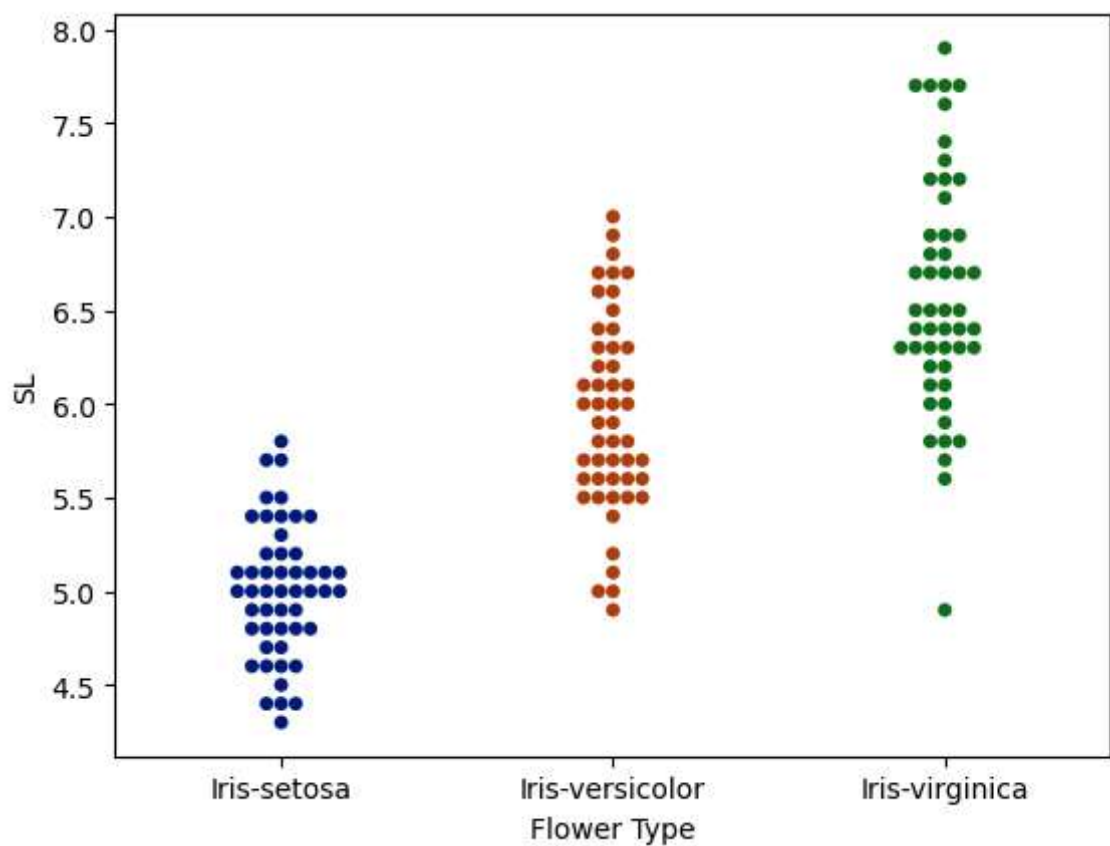
```
In [35]: sns.violinplot(y = iris.SL,x = iris['Flower Type'],palette='crest')  
plt.show()
```



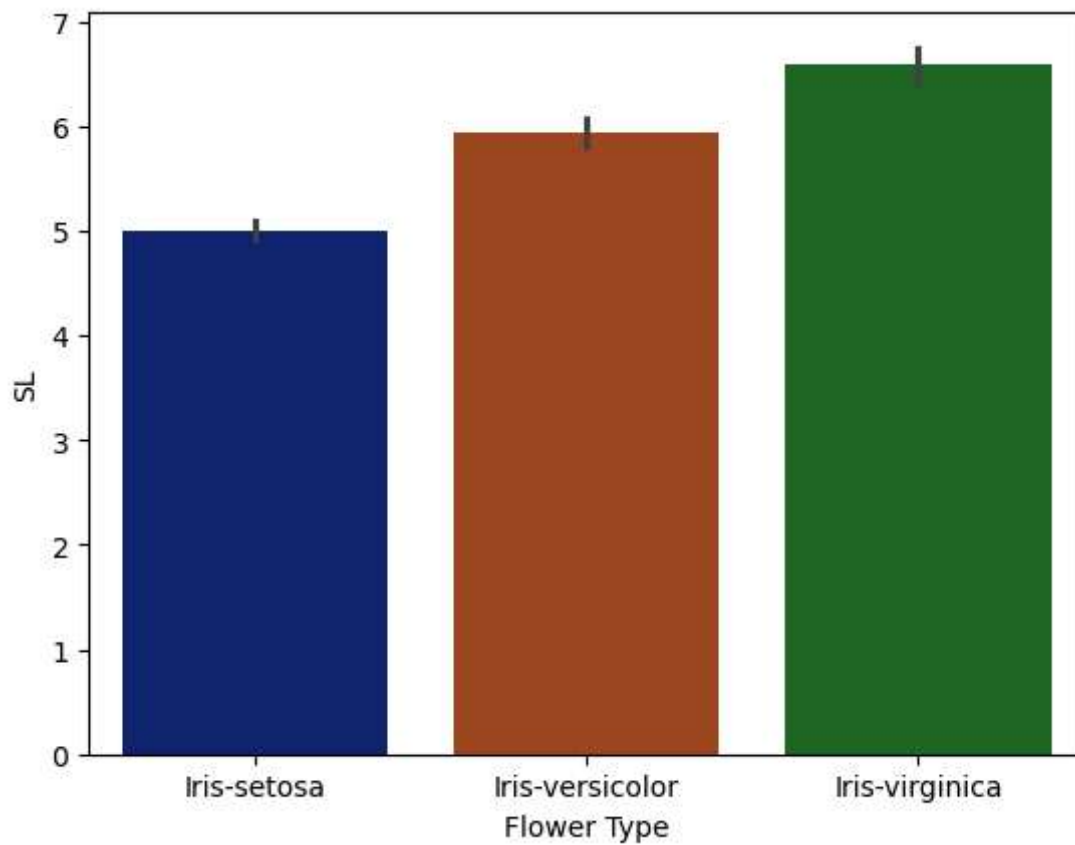
```
In [36]: sns.stripplot(y = iris.SL,x = iris['Flower Type'],palette='dark')  
plt.show()
```



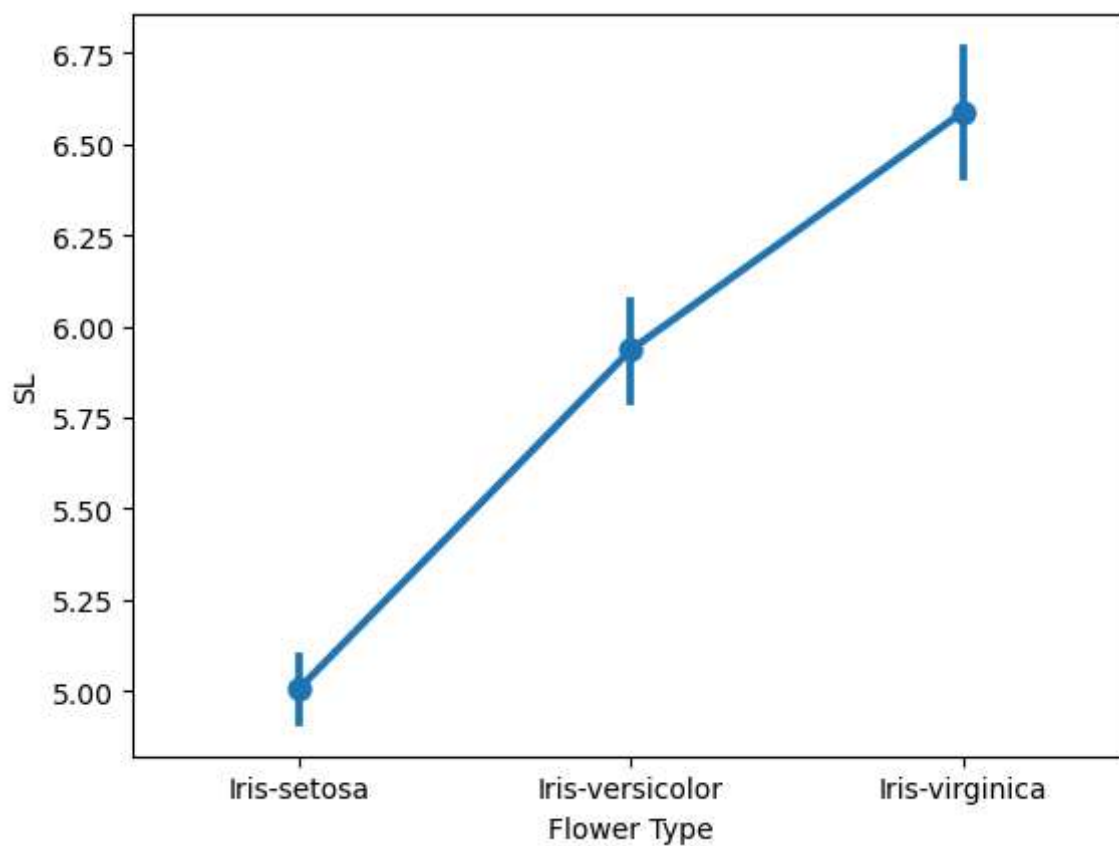
```
In [37]: sns.swarmplot(y = iris.SL,x = iris['Flower Type'],palette='dark')  
plt.show()
```



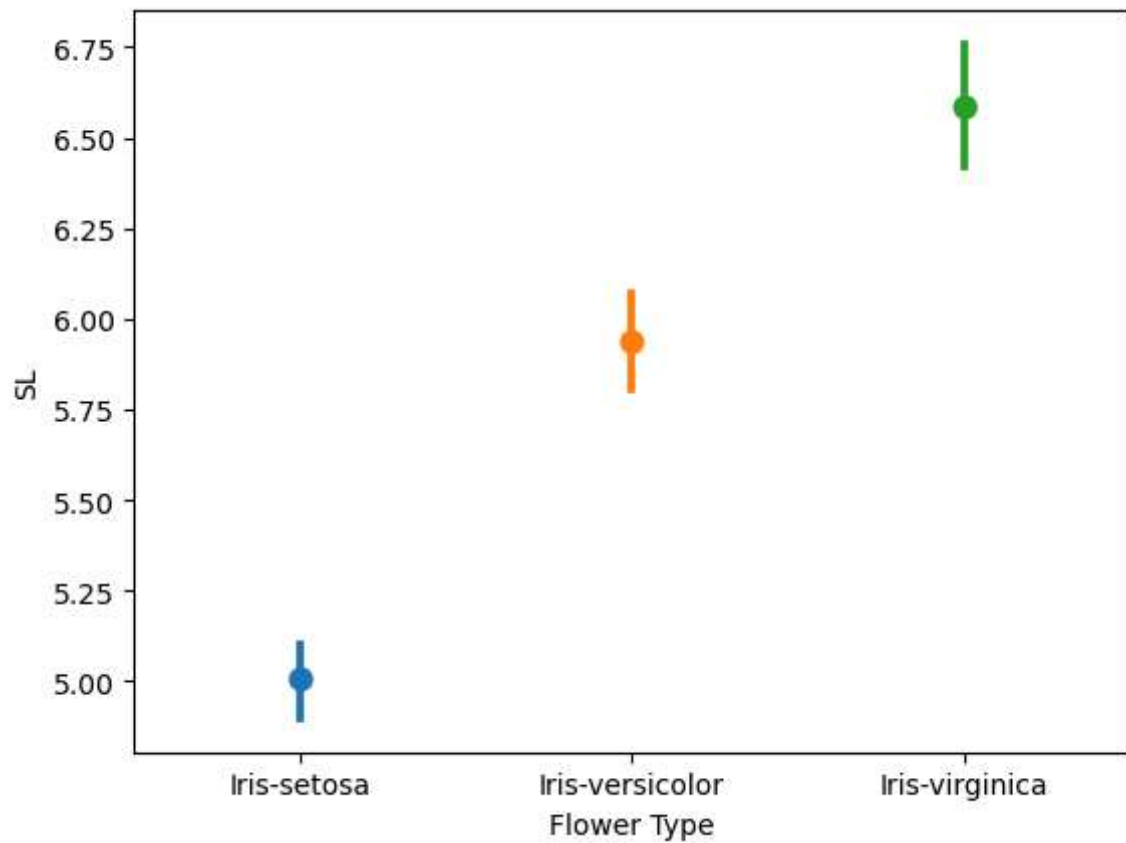
```
In [38]: sns.barplot(y = iris.SL,x = iris['Flower Type'],palette='dark')  
plt.show()
```



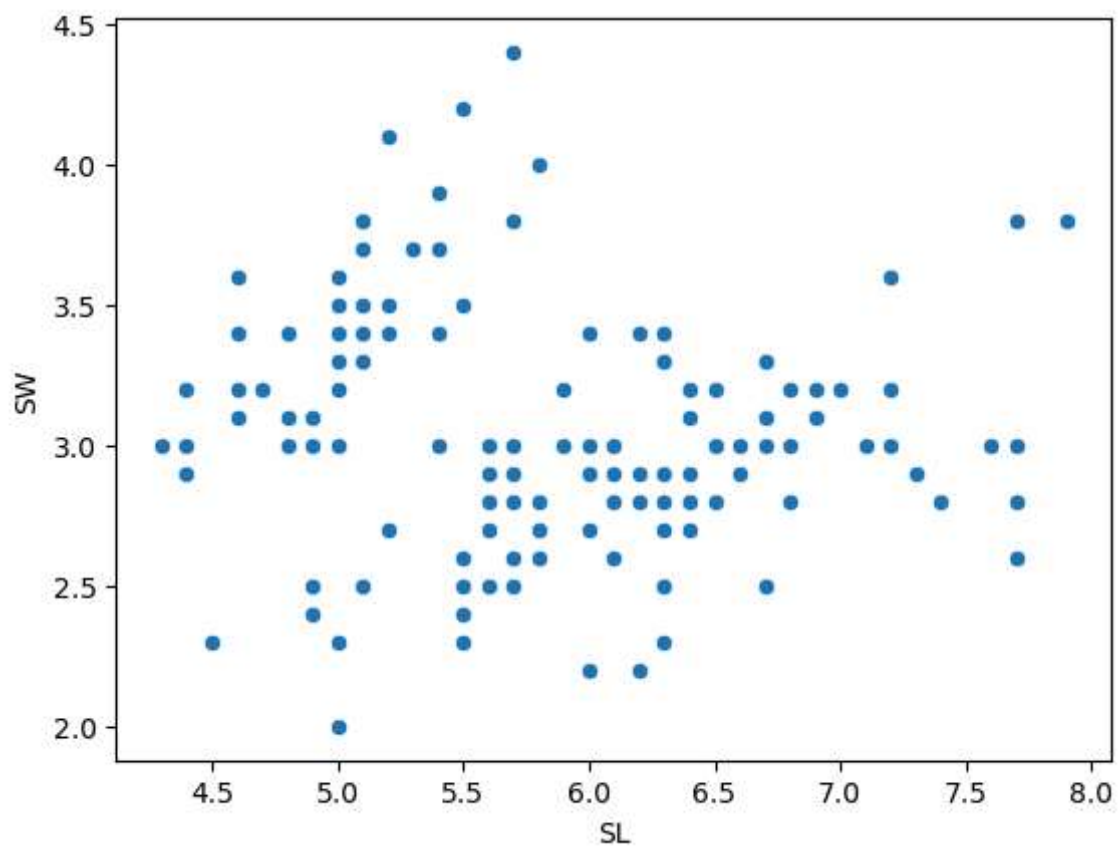
```
In [39]: sns.pointplot(y = iris.SL, x = iris['Flower Type'])  
plt.show()
```



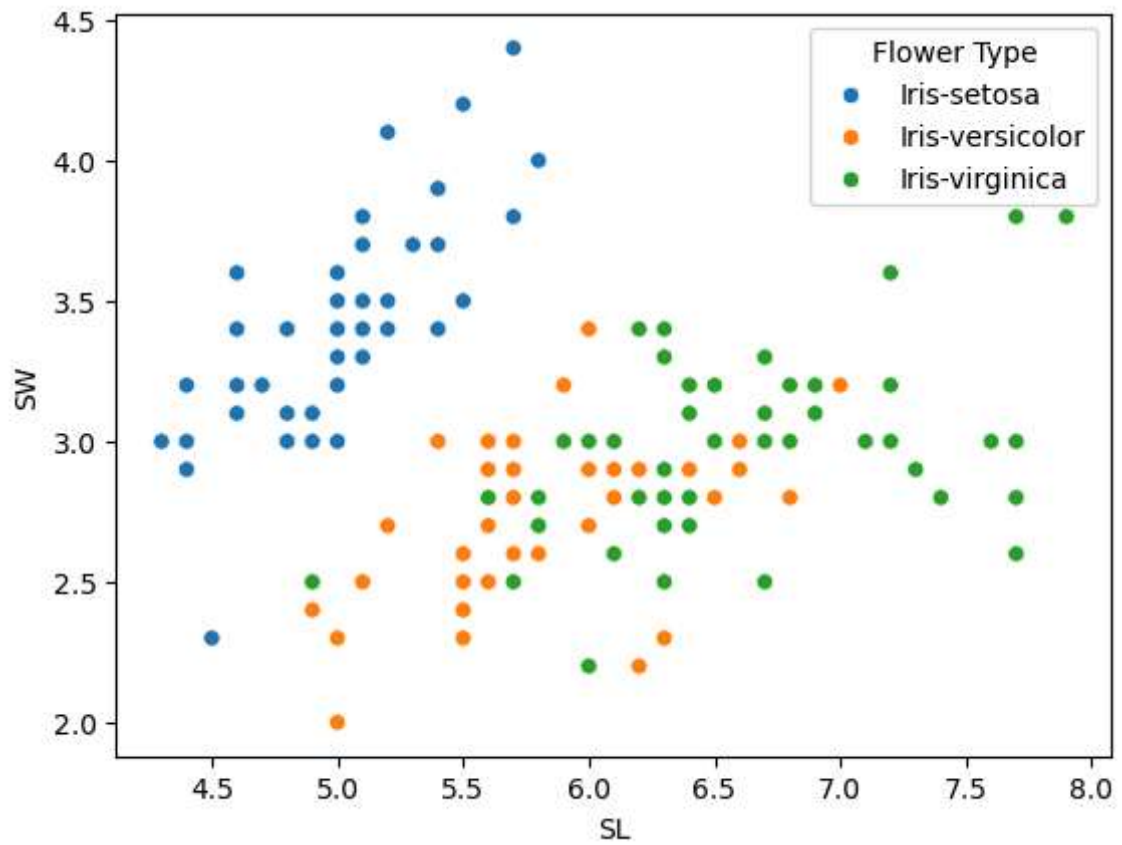
```
In [40]: sns.pointplot(y = iris.SL, x = iris['Flower Type'], hue=iris['Flower Type'])  
plt.show()
```



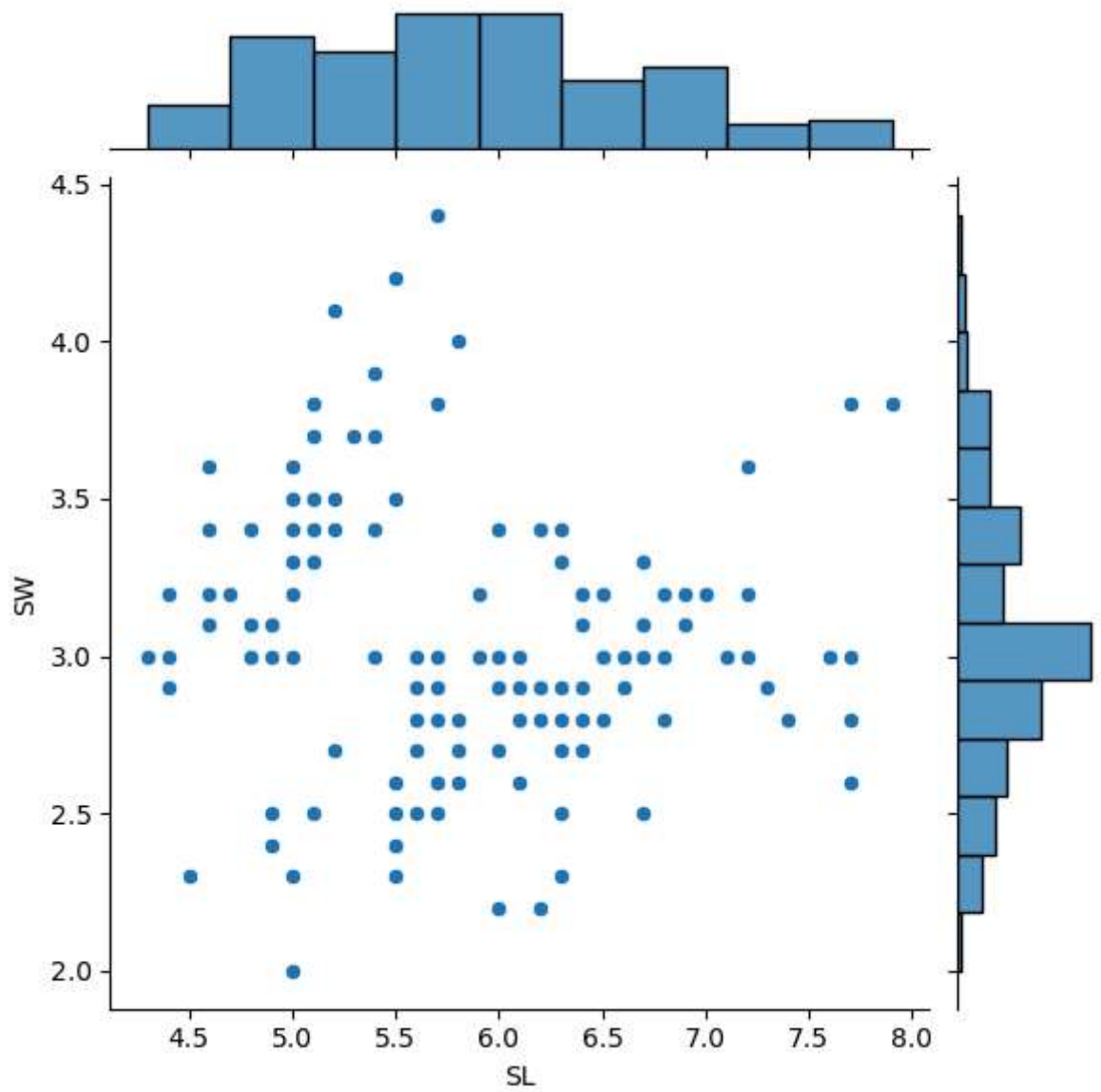
```
In [41]: sns.scatterplot(x = iris.SL, y = iris.SW)
plt.show()
```



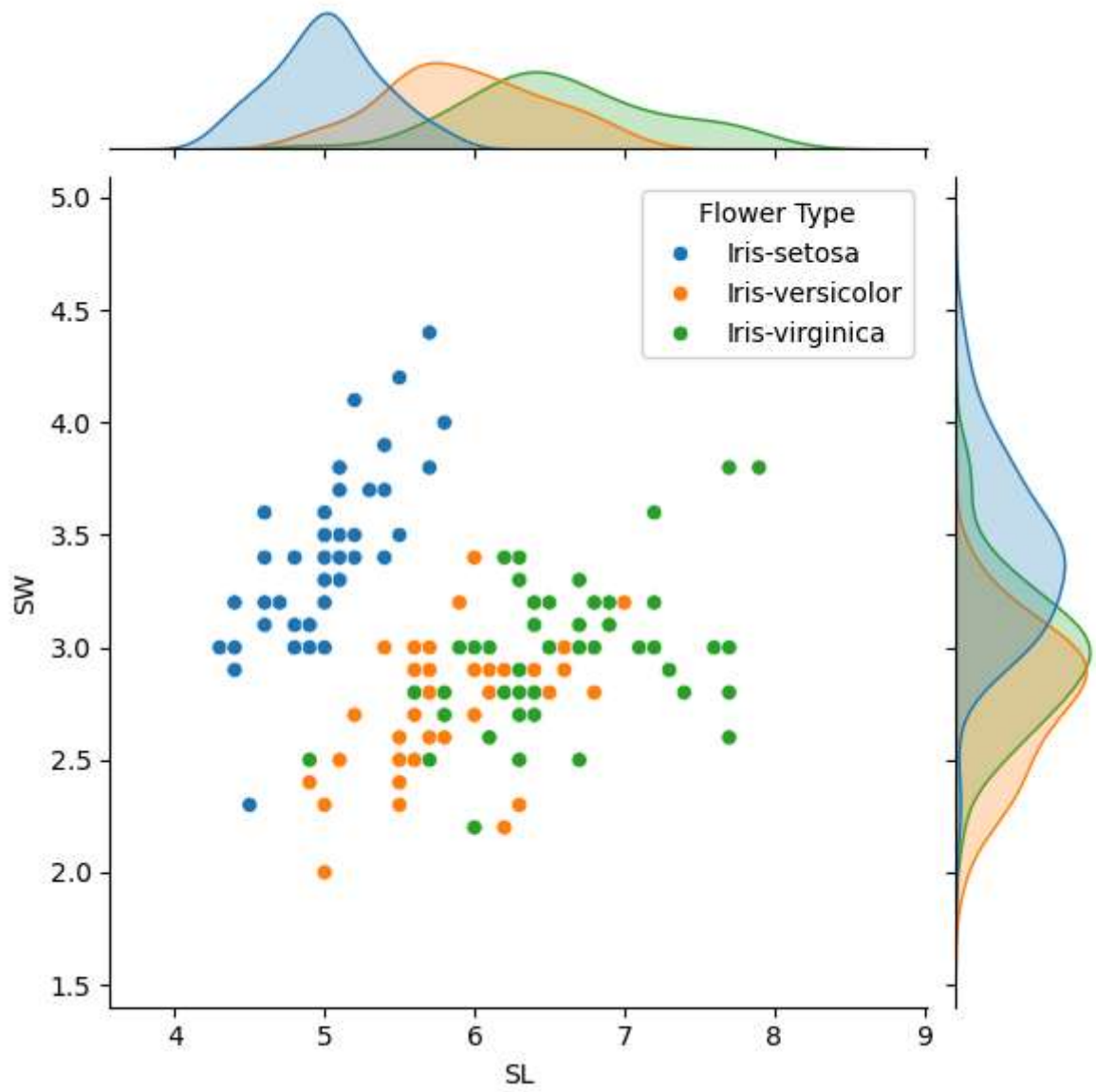
```
In [42]: sns.scatterplot(x = iris.SL, y = iris.SW, hue=iris['Flower Type'])
plt.show()
```

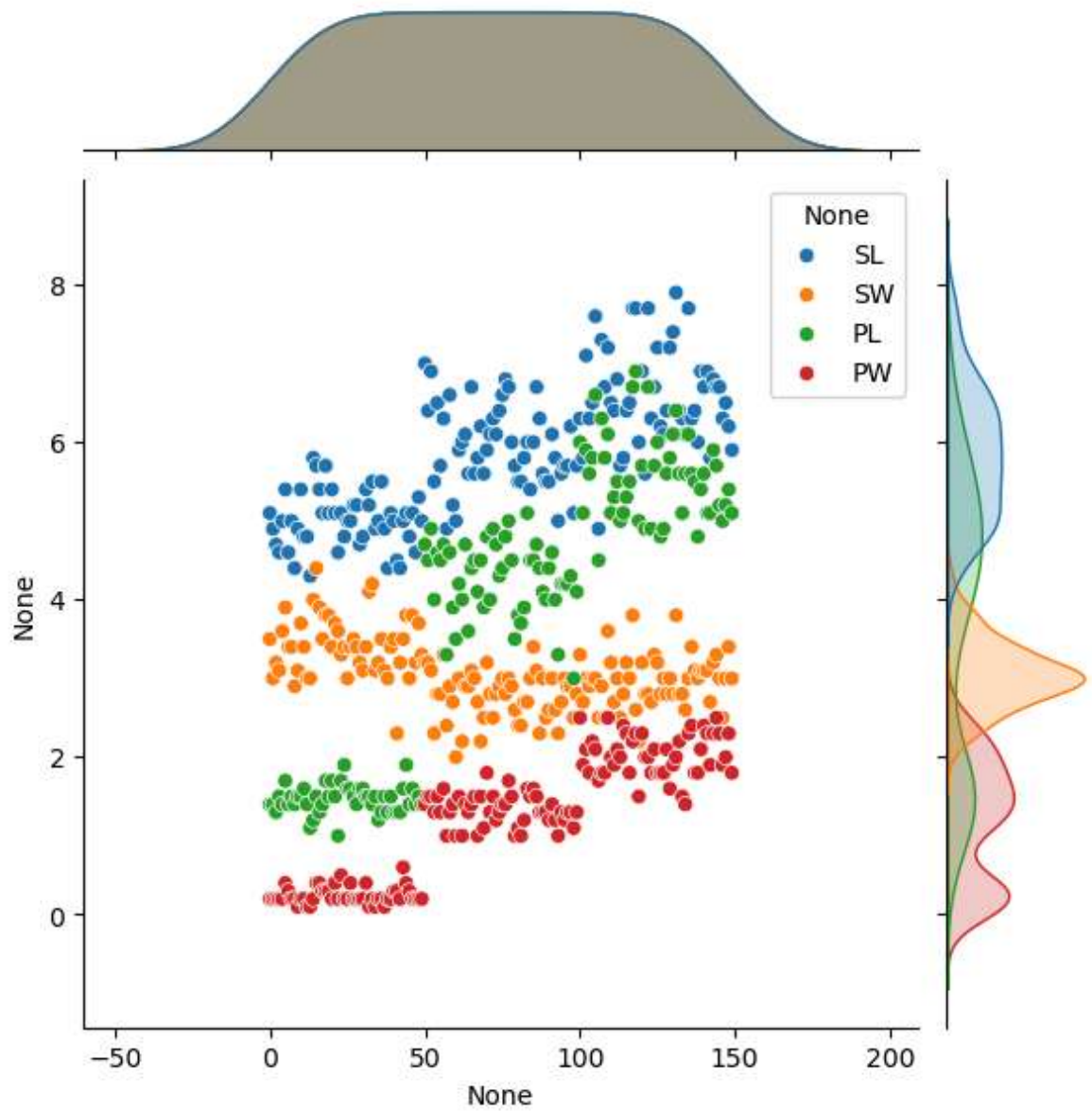
```
In [43]: sns.jointplot(x = iris.SL, y = iris.SW)
plt.show()
```



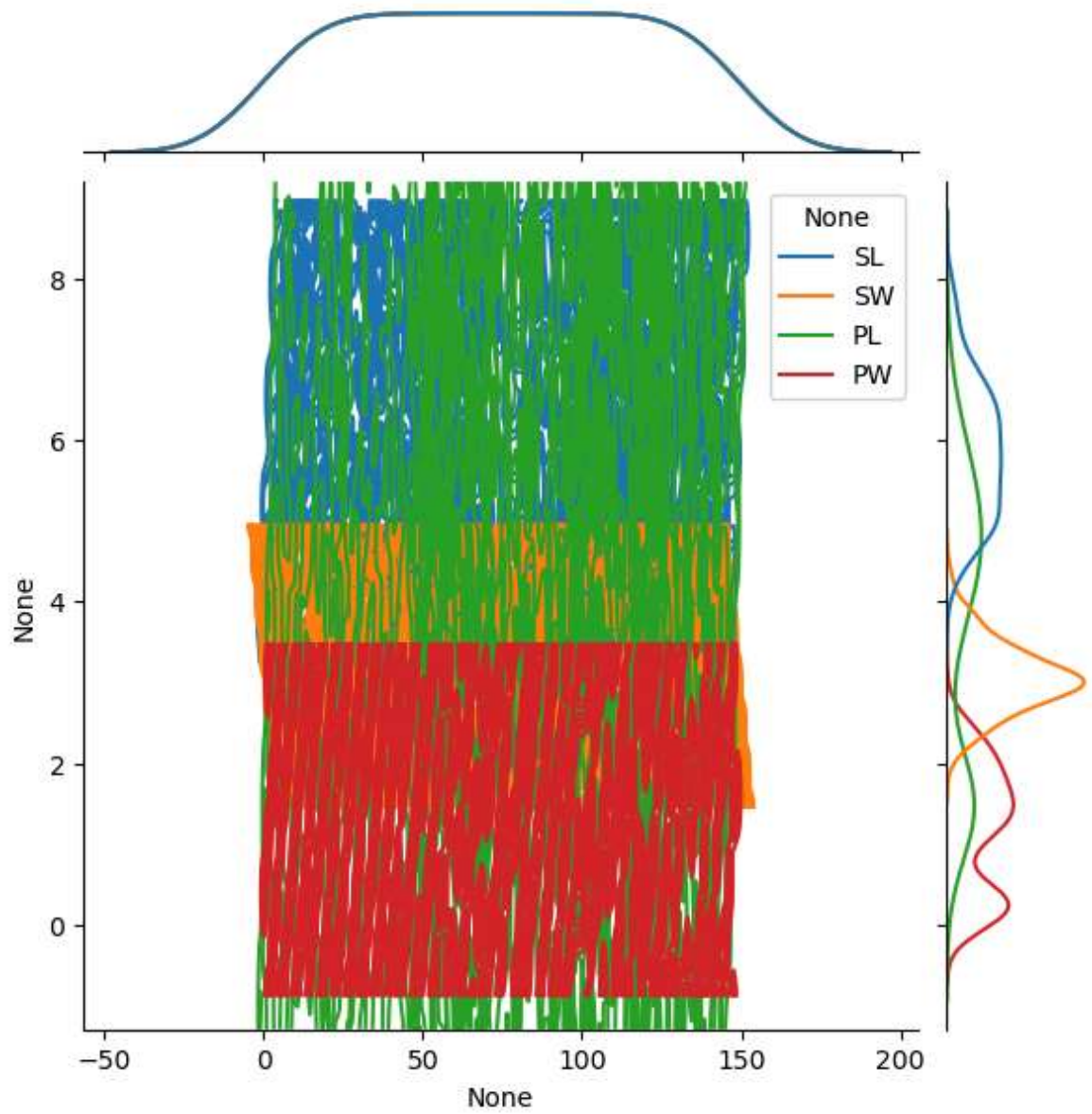
```
In [44]: sns.jointplot(x = iris.SL, y = iris.SW, hue=iris['Flower Type'])  
plt.show()
```



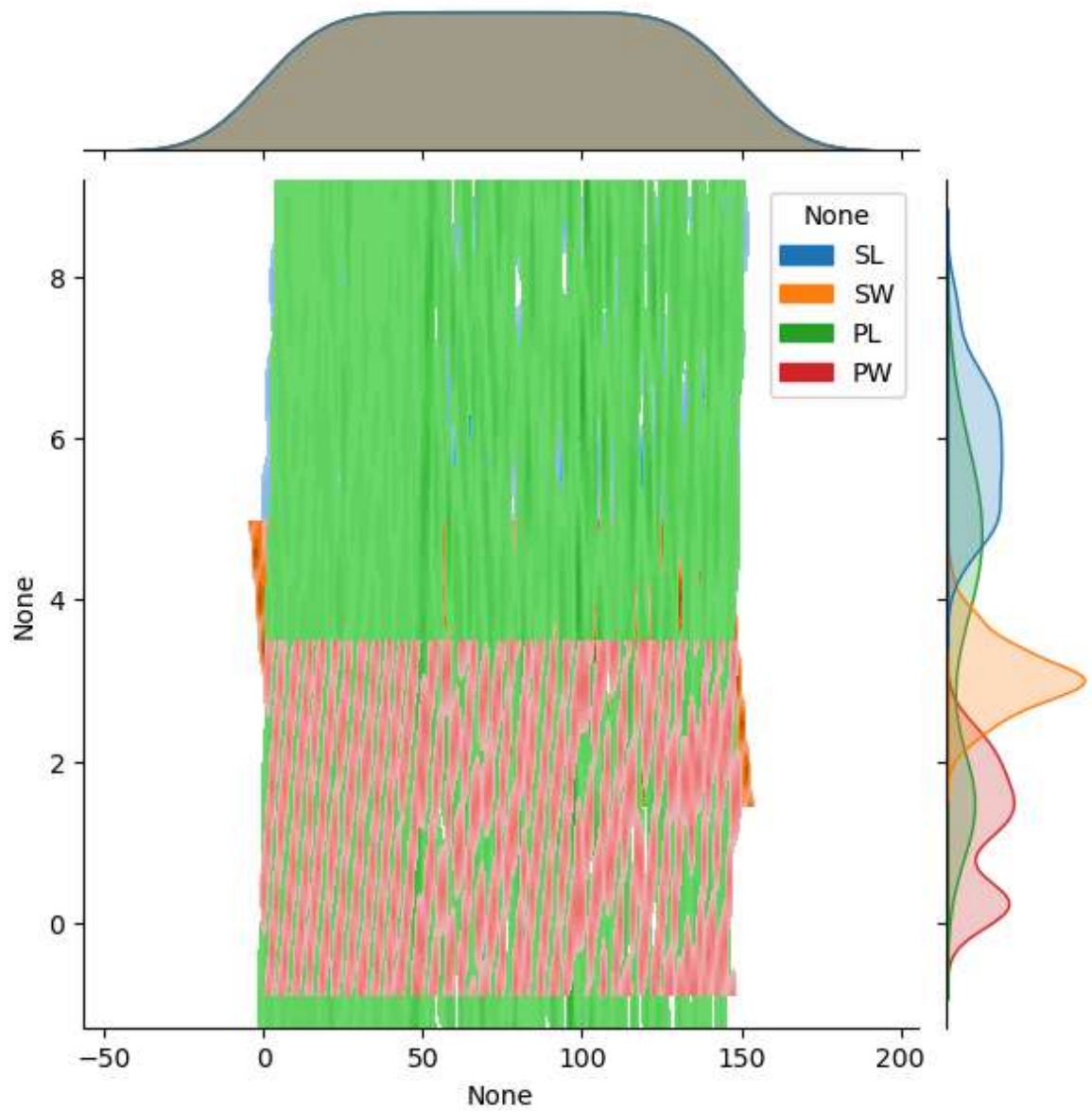
```
In [45]: sns.jointplot(data=iris)
plt.show()
```



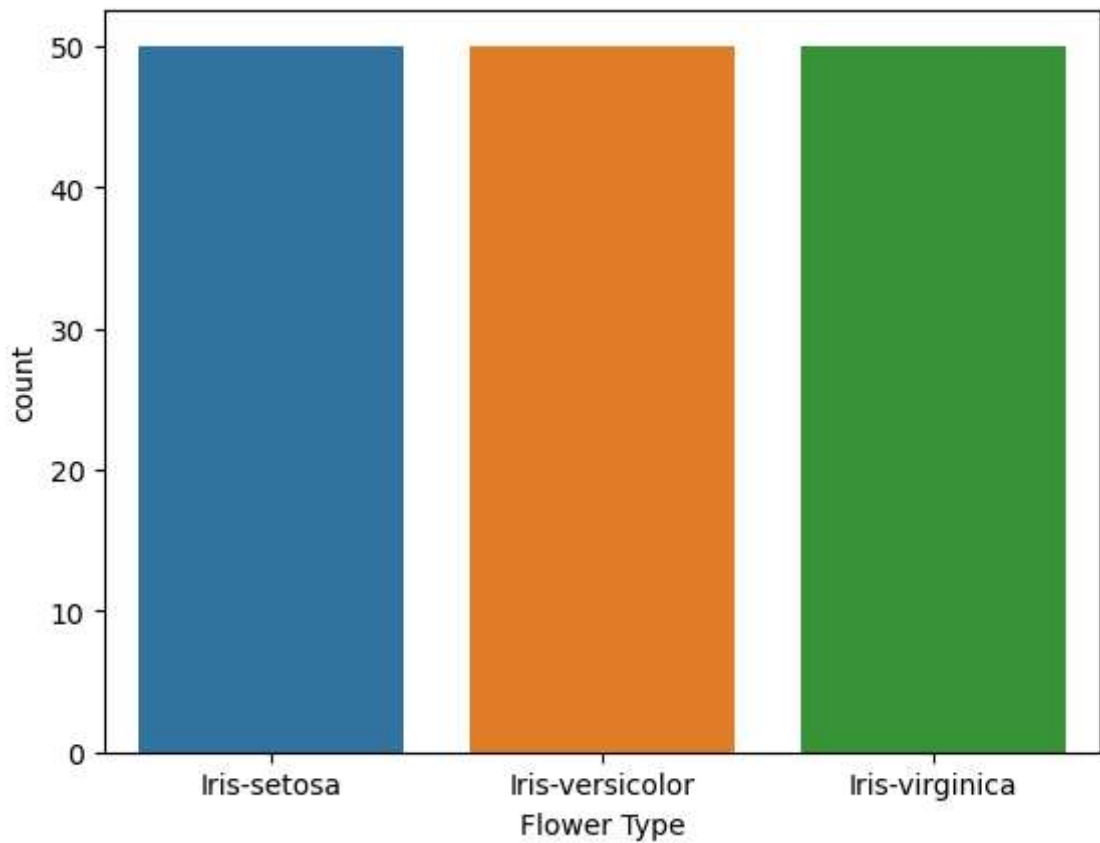
```
In [46]: sns.jointplot(data=iris,kind='kde')  
plt.show()
```



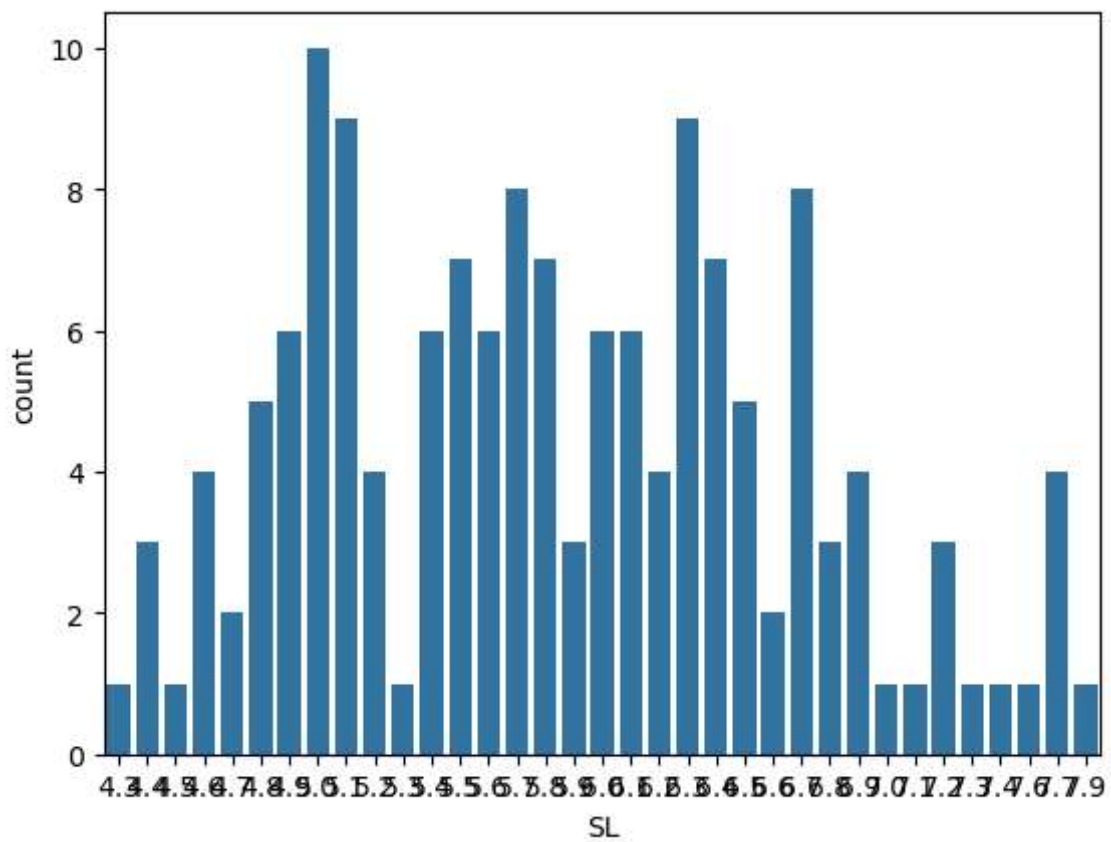
```
In [47]: sns.jointplot(data= iris,kind='kde',fill=True)  
plt.show()
```



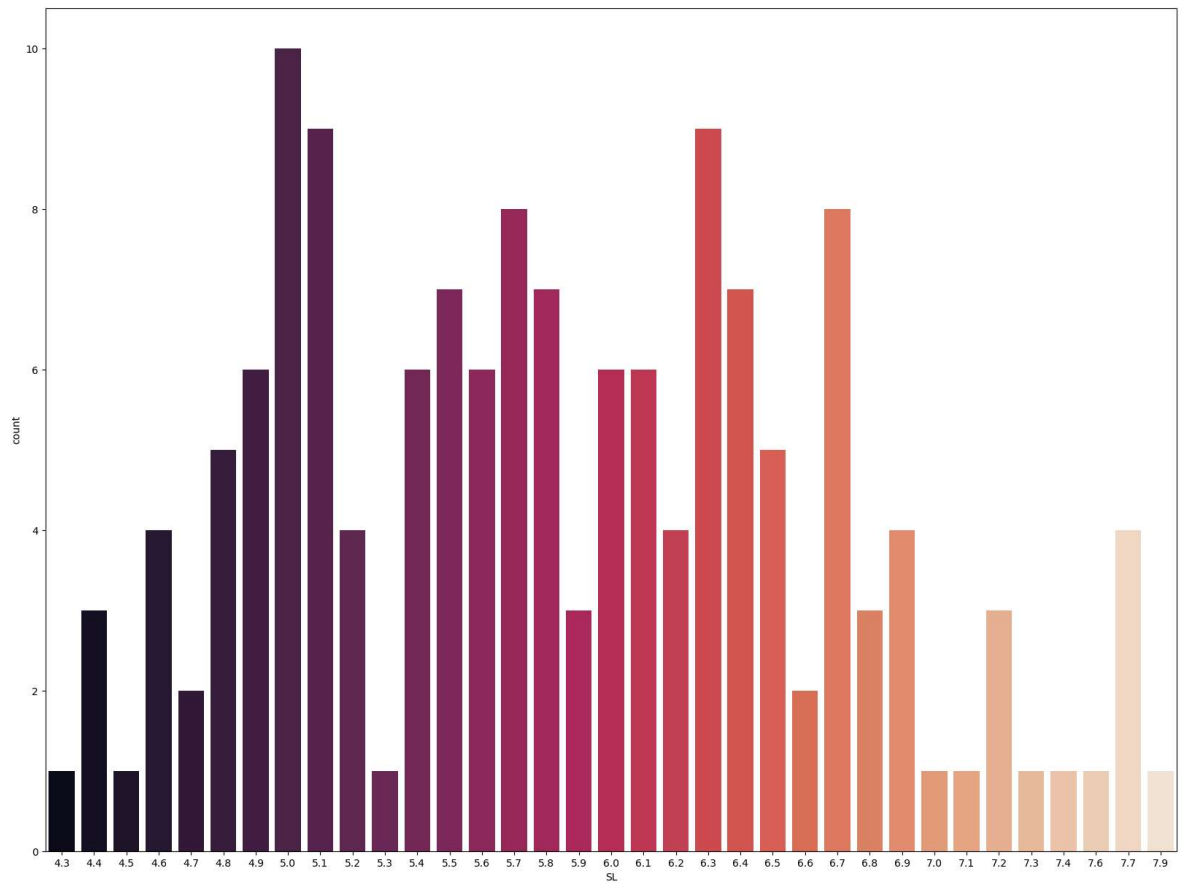
```
In [48]: sns.countplot(x=iris['Flower Type'],hue=iris['Flower Type'])  
plt.show()
```



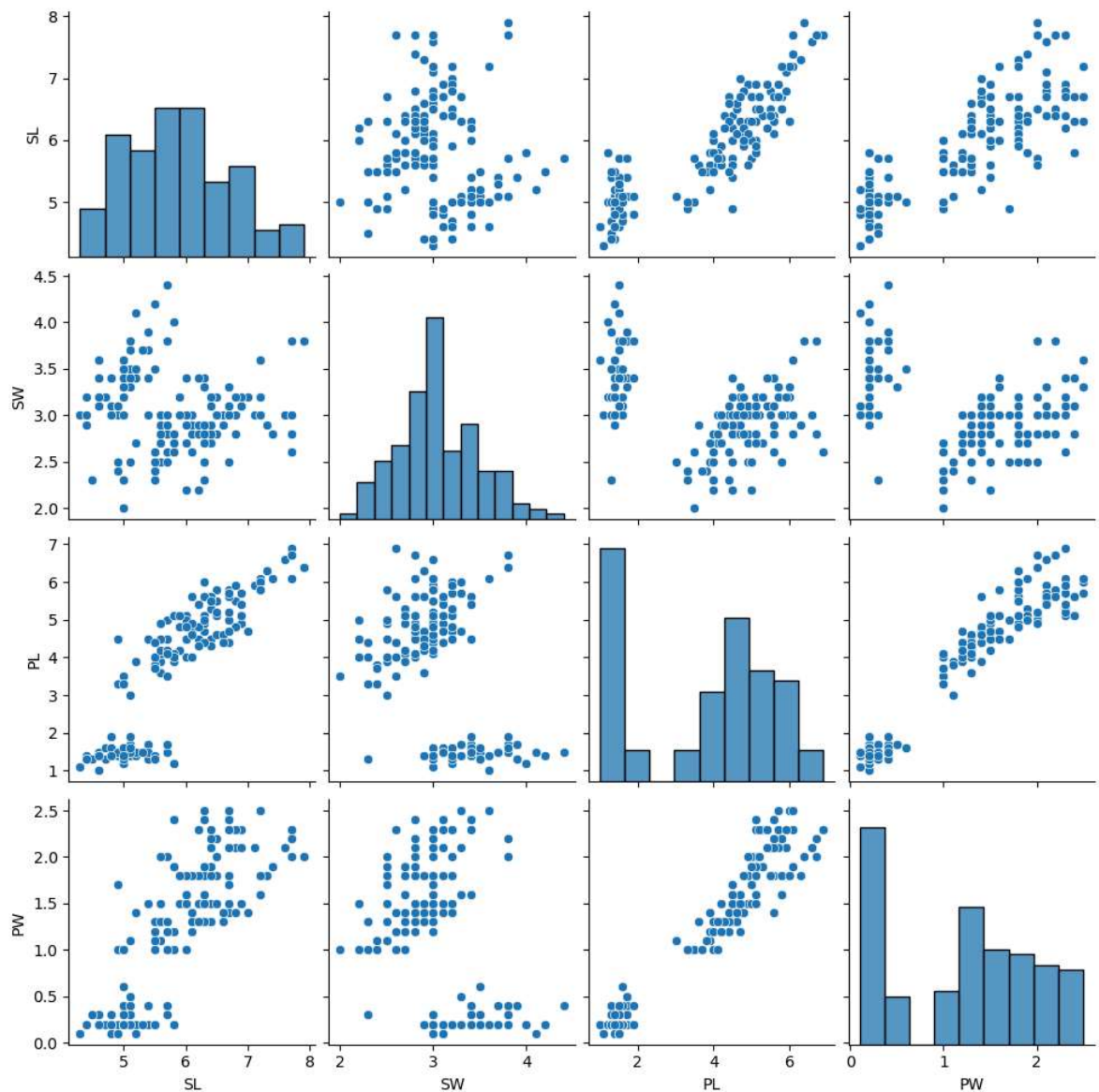
```
In [49]: sns.countplot(x=iris.SL)  
plt.show()
```



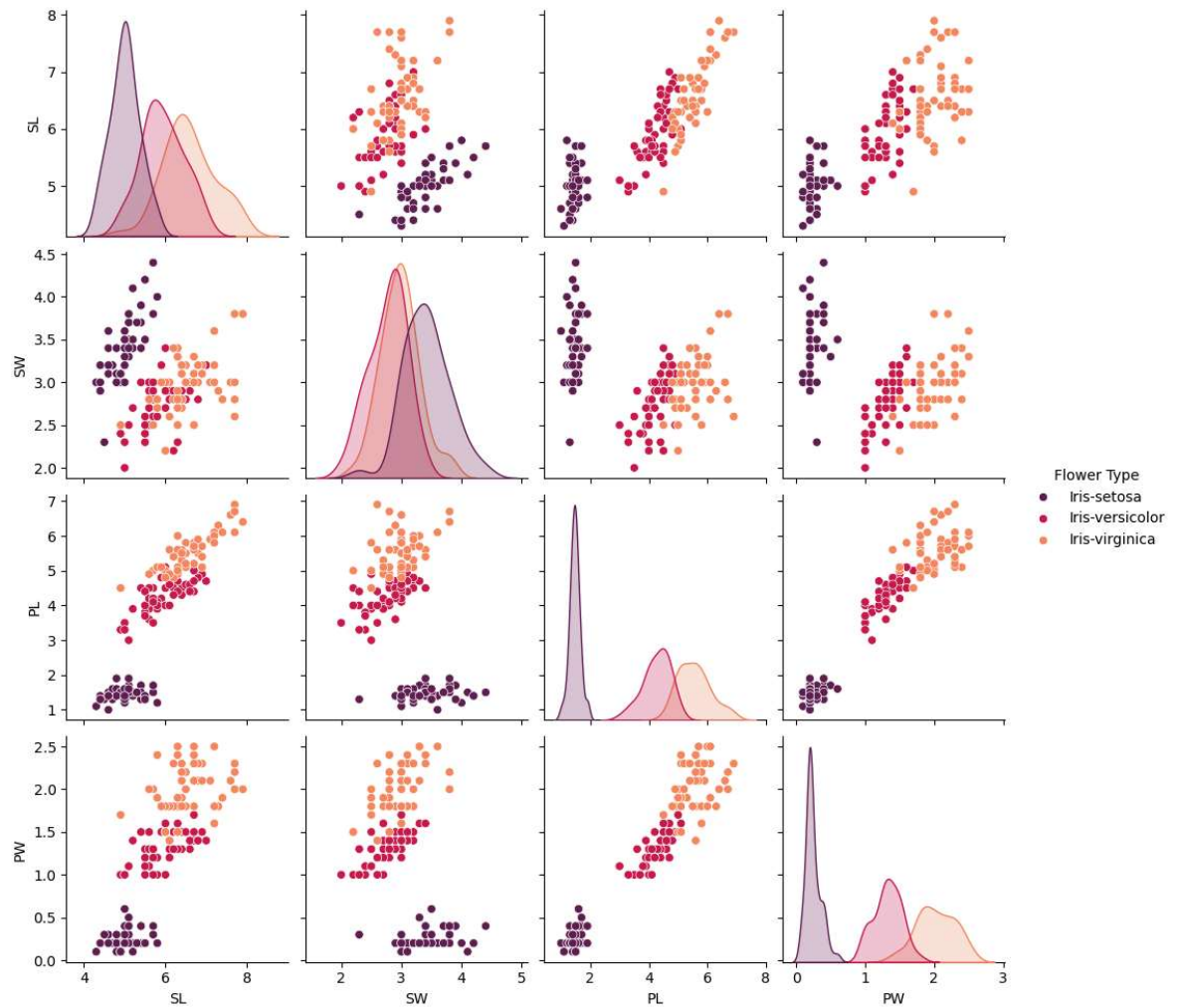
```
In [50]: plt.figure(figsize=(20,15))  
sns.countplot(x=iris.SL,palette='rocket')  
plt.show()
```



```
In [51]: sns.pairplot(iris)
plt.show()
```

```
In [52]: sns.pairplot(iris,hue='Flower Type',palette='rocket')  
plt.show()
```



```
In [53]: a = iris.corr()  
a
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[53], line 1
----> 1 a = iris.corr()
      2 a

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:11049, in DataFrame.corr(
self, method, min_periods, numeric_only)
    11047 cols = data.columns
    11048 idx = cols.copy()
-> 11049 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
    11051 if method == "pearson":
    11052     correl = libalgos.nancorr(mat, minp=min_periods)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:1993, in DataFrame.to_numpy(
self, dtype, copy, na_value)
    1991 if dtype is not None:
    1992     dtype = np.dtype(dtype)
-> 1993 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    1994 if result.dtype is not dtype:
    1995     result = np.asarray(result, dtype=dtype)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1694, in BlockManager.as_array(self, dtype, copy, na_value)
    1692     arr.flags.writeable = False
    1693 else:
-> 1694     arr = self._interleave(dtype=dtype, na_value=na_value)
    1695     # The underlying data was copied within _interleave, so no need
    1696     # to further copy if copy=True or setting na_value
    1698 if na_value is lib.no_default:

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1753, in BlockManager._interleave(self, dtype, na_value)
    1751     else:
    1752         arr = blk.get_values(dtype)
-> 1753     result[rl.indexer] = arr
    1754     itemmask[rl.indexer] = 1
    1756 if not itemmask.all():

ValueError: could not convert string to float: 'Iris-setosa'

```

```

In [54]: iris.drop('Flower Type',axis=1,inplace=True)
iris

```

Out[54]:

	SL	SW	PL	PW
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

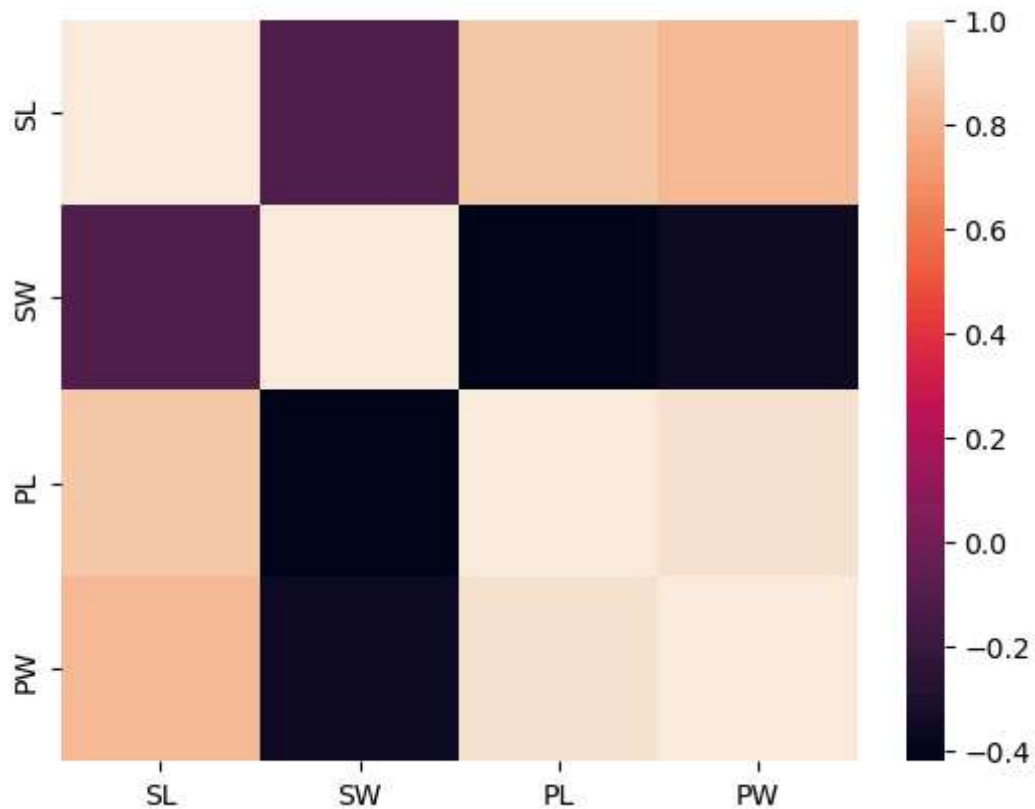
150 rows × 4 columns

```
In [55]: C = iris.corr()
C
```

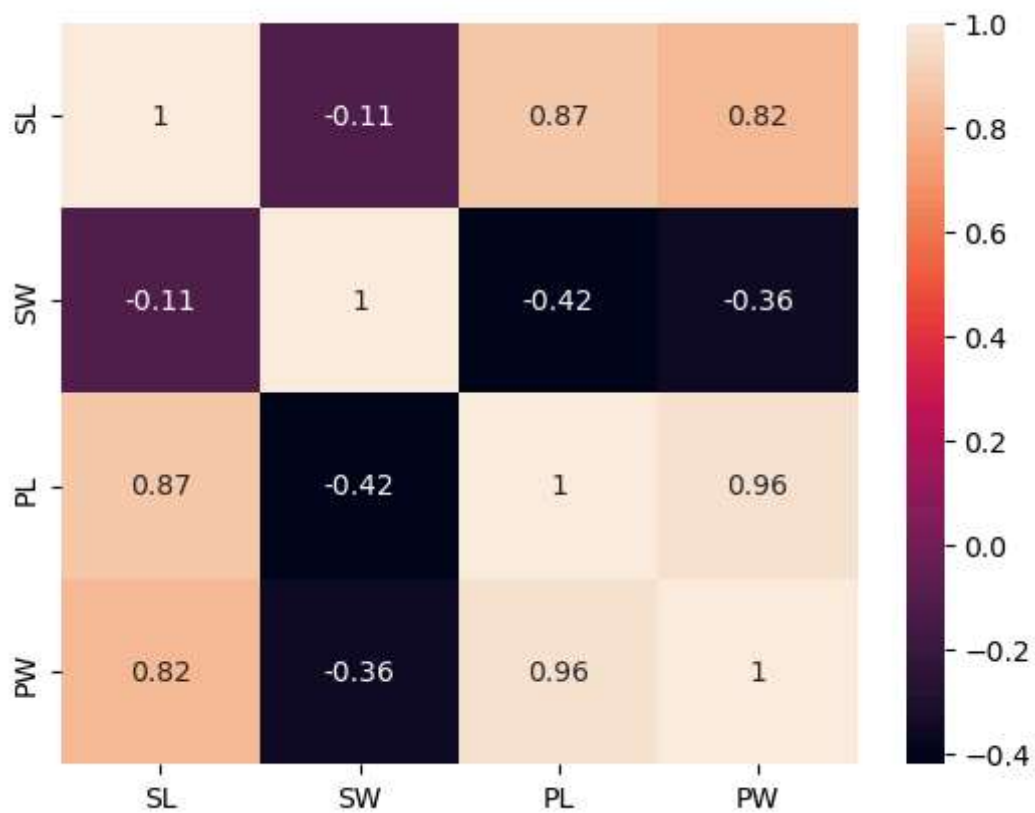
Out[55]:

	SL	SW	PL	PW
SL	1.000000	-0.109369	0.871754	0.817954
SW	-0.109369	1.000000	-0.420516	-0.356544
PL	0.871754	-0.420516	1.000000	0.962757
PW	0.817954	-0.356544	0.962757	1.000000

```
In [56]: sns.heatmap(C)
plt.show()
```



```
In [57]: sns.heatmap(C,annot=C)
plt.show()
```



```
In [61]: sns.heatmap(b,annot=b,cmap='Blues')
plt.show()
```

```

-----
IndexError                                Traceback (most recent call last)
Cell In[61], line 1
----> 1 sns.heatmap(b,annot=b,cmap='Blues')
      2 plt.show()

File ~\anaconda3\Lib\site-packages\seaborn\matrix.py:446, in heatmap(data, vmin,
vmax, cmap, center, robust, annot, fmt, annot_kws, linewidths, linecolor, cbar, c
bar_kws, cbar_ax, square, xticklabels, yticklabels, mask, ax, **kwargs)
    365 """Plot rectangular data as a color-encoded matrix.
    366
    367 This is an Axes-level function and will draw the heatmap into the
    (...)
    443
    444 """
    445 # Initialize the plotter object
--> 446 plotter = _HeatMapper(data, vmin, vmax, cmap, center, robust, annot, fmt,
    447                        annot_kws, cbar, cbar_kws, xticklabels,
    448                        yticklabels, mask)
    450 # Add the pcolormesh kwargs here
    451 kwargs["linewidths"] = linewidths

File ~\anaconda3\Lib\site-packages\seaborn\matrix.py:115, in _HeatMapper.__init__
(self, data, vmin, vmax, cmap, center, robust, annot, fmt, annot_kws, cbar, cbar_
kws, xticklabels, yticklabels, mask)
    112 # Validate the mask and convert to DataFrame
    113 mask = _matrix_mask(data, mask)
--> 115 plot_data = np.ma.masked_where(np.asarray(mask), plot_data)
    117 # Get good names for the rows and columns
    118 xtickevery = 1

File ~\anaconda3\Lib\site-packages\numpy\ma\core.py:1933, in masked_where(conditi
on, a, copy)
    1931 (cshape, ashape) = (cond.shape, a.shape)
    1932 if cshape and cshape != ashape:
-> 1933     raise IndexError("Inconsistent shape between the condition and the in
put")
    1934             " (got %s and %s)" % (cshape, ashape))
    1935 if hasattr(a, '_mask'):
    1936     cond = mask_or(cond, a._mask)

IndexError: Inconsistent shape between the condition and the input (got (4, 1) an
d (4,))

```

```
In [60]: #!pip install seaborn --upgrade
```

```
In [ ]:
```